Introduction to Artificial Intelligence

Chapter 4 - Part I: Local Search and Continuous Spaces

Namyup Kim

Soongsil University



Local Search: What It Is

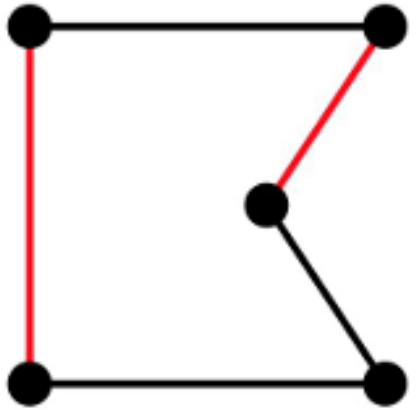
- Local search: works from the current state and moves to a neighboring state
  - It does not keep the path, and the sequence of states reached does not matter
- Examples: the N-queens problem, integrated-circuit design, factory floor layout, network optimization, crop planning
  - It is not systematic, so it can fail to find a solution at all ☹
  - It uses very little memory ☺
  - It can find a reasonable solution even in an infinite (continuous) state space ☺
  - It is well suited to optimization problems — find the best state by an objective function ☺



Local Search as Iterative Improvement

- In many optimization problems the path is irrelevant; the goal state is itself the solution.
- Then state space = set of "complete" configurations;
  find **configuration satisfying constraints**, e.g., n-queens problem;
  or, find **optimal configuration**, e.g., travelling salesperson problem





- In such cases use an *iterative improvement* algorithm: keep a single current state and try to improve it.

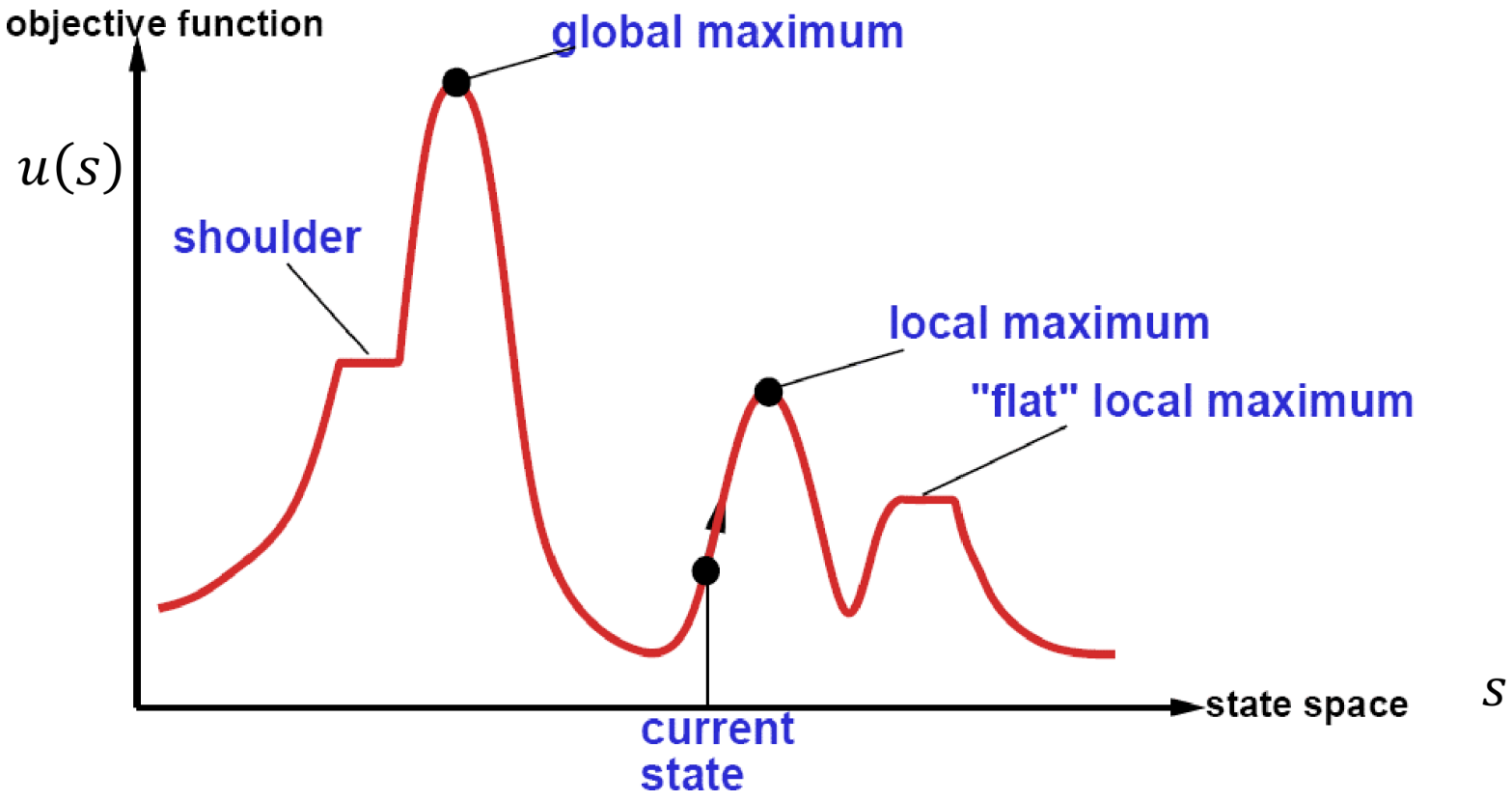
How Local Search Differs from Tree Search

1. Each state's utility (*score*) is known or can be computed. The goal is to find a state with highest utility
2. Often there is no explicit initial state, and neither the path to the goal nor its cost matters
3. No search tree is used. Only the current state is stored, and the agent moves to a better state when it can — almost no memory, and very simple code
   - **Constant space complexity (memory)**, which suits online search as well as offline search.

	Memory usage	Space complexity
BFS	Frontier + reached table	$O(b^d)$
A*	Frontier + reached table	$O(b^d)$
DFS/IDS	Current path + its siblings	$O(bd)$
Local Search	Current	constant

The State-Space Landscape

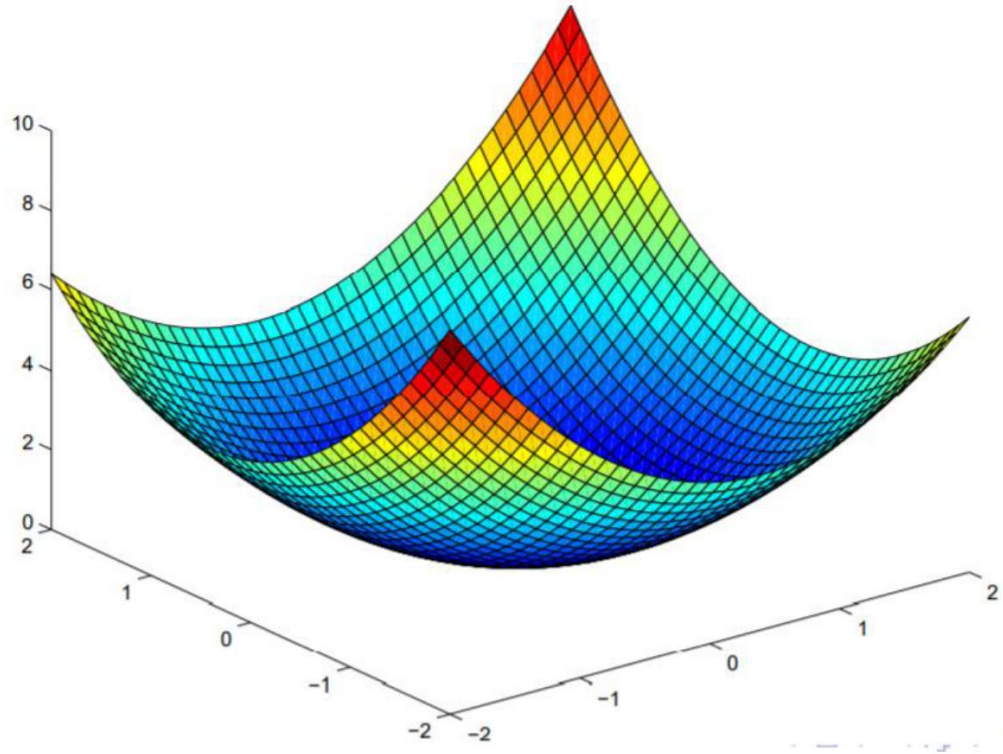
- Treat the objective function as elevation. Then the goal of local search is the global maximum, and climbing towards it is hill climbing
- How do we escape a local maximum?
  - one answer is to start "hill climbing" again *from a random state*



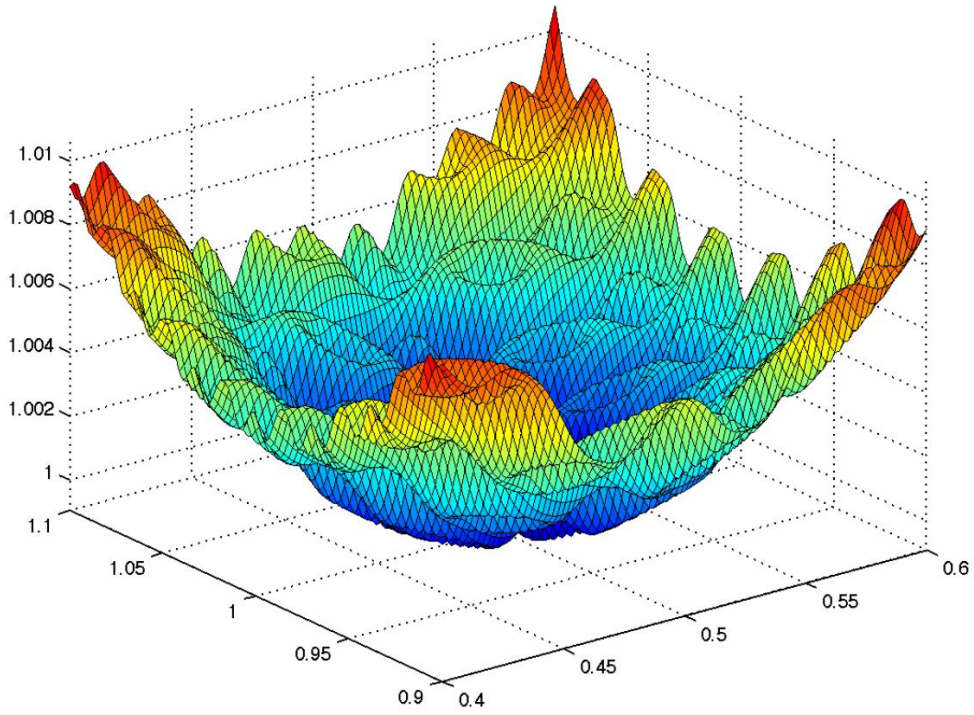
Convex and Non-Convex Optimisation

- Convex: one global optimum, and a smooth continuous function
  - Solve it by differentiating, $f'(x) = 0$
- Non-convex: many local optima, so the solution is hard to find

Convex Problem



Non-convex Problem



Hill-Climbing Search, or Greedy Local Search

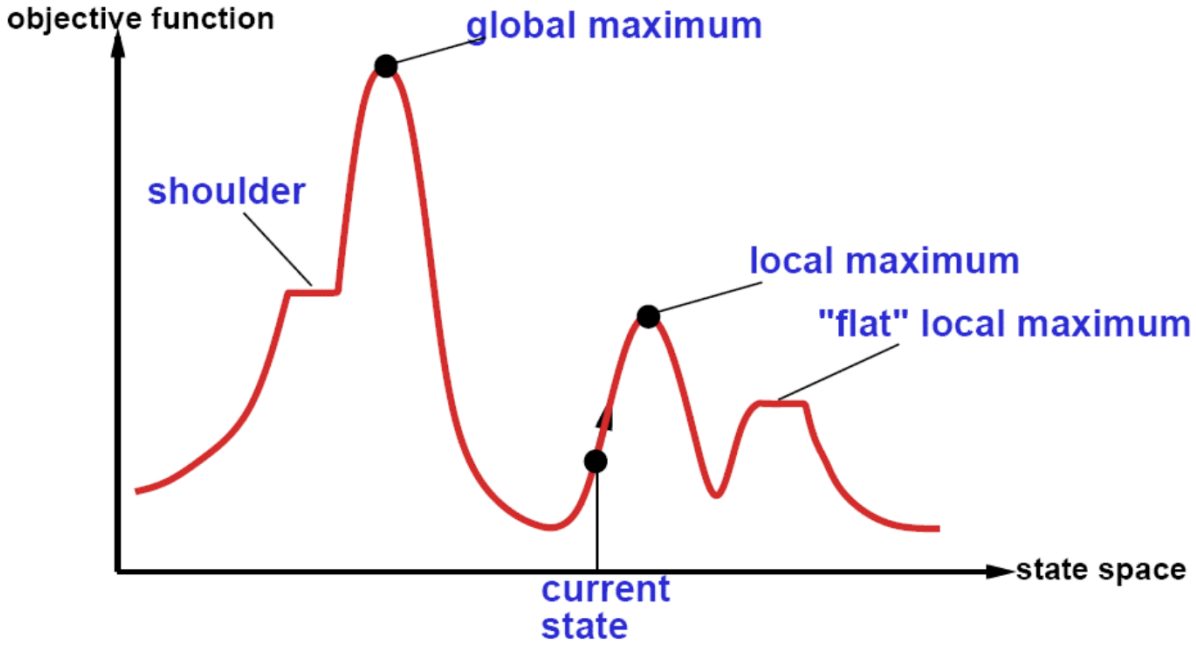
- A simple local search, general idea:
  - Start wherever.
  - Repeat: move to the best neighboring state.
  - If no neighbor is better than the current state, quit.
- Greedy in the sense of Chapter 3's greedy best-first search
  - It follows the objective function alone and never looks ahead
- It keeps only the current state. There is no frontier

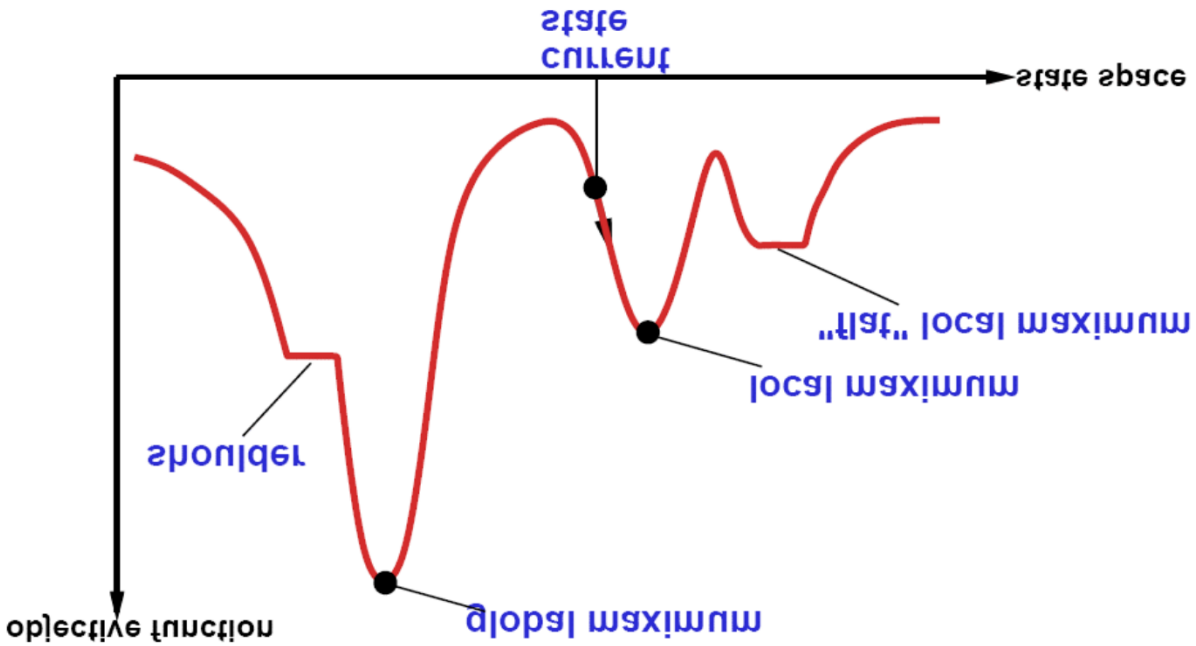


Minimization and Maximization Are the Same Problem

- In AI, hill climbing means walking uphill towards the maximum of a function
  - In practice the problem is more often stated as a minimization
- The two are the same: $\max f(x)$ is $\min -f(x)$

$$\max\big(f(x)\big) \quad \Leftrightarrow \quad \min\big(-f(x)\big)$$





Hill-Climbing Search: The Algorithm

Maximization

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← problem.INITIAL
    while true do
        neighbor ← a highest-valued successor state of current
        if VALUE(neighbor) ≤ VALUE(current) then return current
        current ← neighbor
```

Random initialization in general

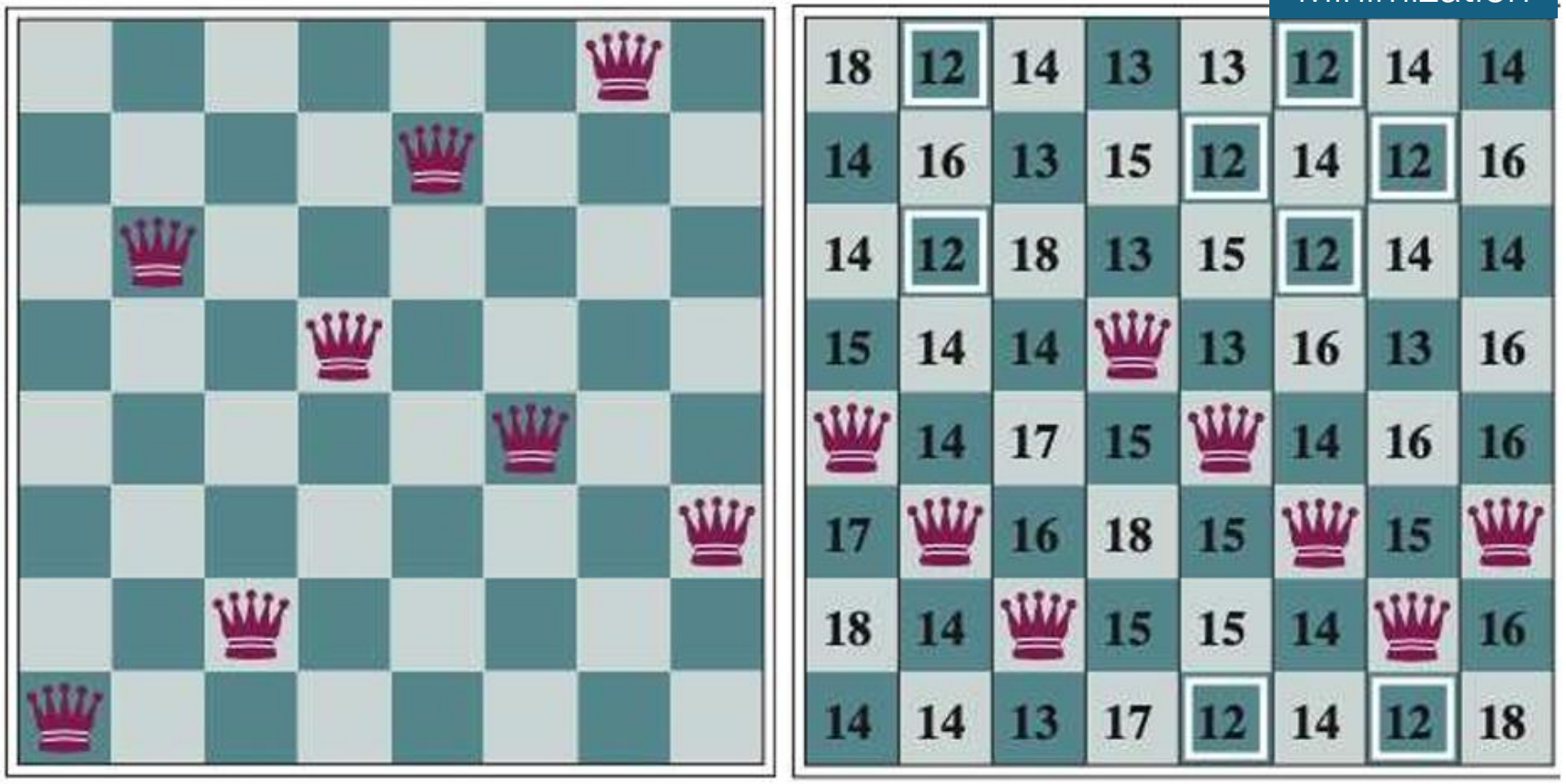
≥ for minimization

- Variants of the Hill-Climbing Search
  - **Steepest-ascent hill climbing**: look at every successor and take the highest-valued one.
  - **Stochastic hill climbing** : choose at random among the uphill moves (still generate all successors; less greedy).
  - **First-choice stochastic hill climbing**: generate one random successor at a time and move only if it is better.

	Successors generated	Move taken
Steepest-ascent	All	The steepest uphill one
Stochastic	All	A random uphill one
First-choice	One at a time	The first uphill one

A Heuristic for the n-Queens Problem

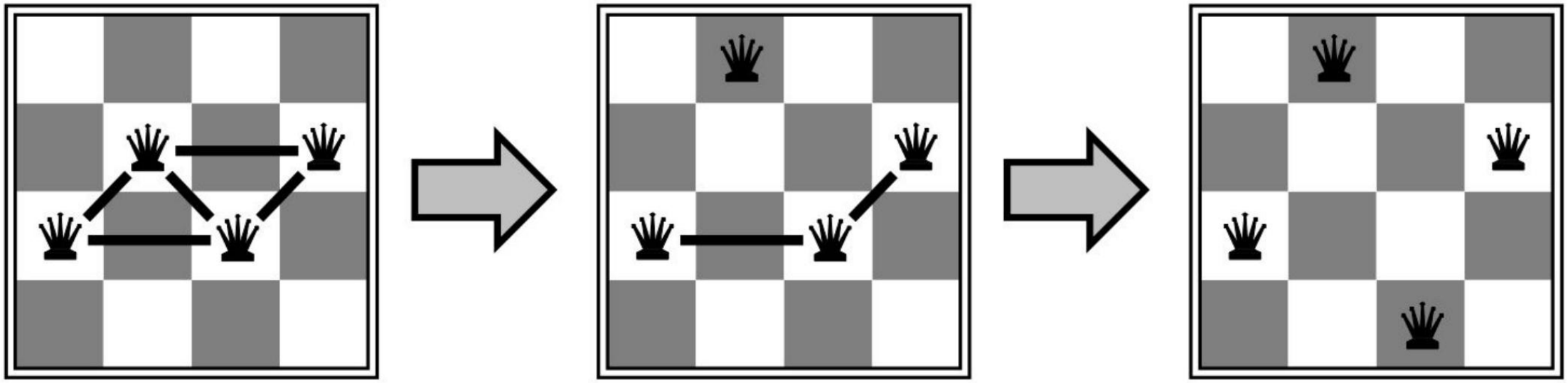
- The 8-queens problem
  - Goal: place 8 queens on a chess board so that no queen attacks another.
  - The board shows the value of h for each possible successor obtained by moving a queen within its column.



h = 17

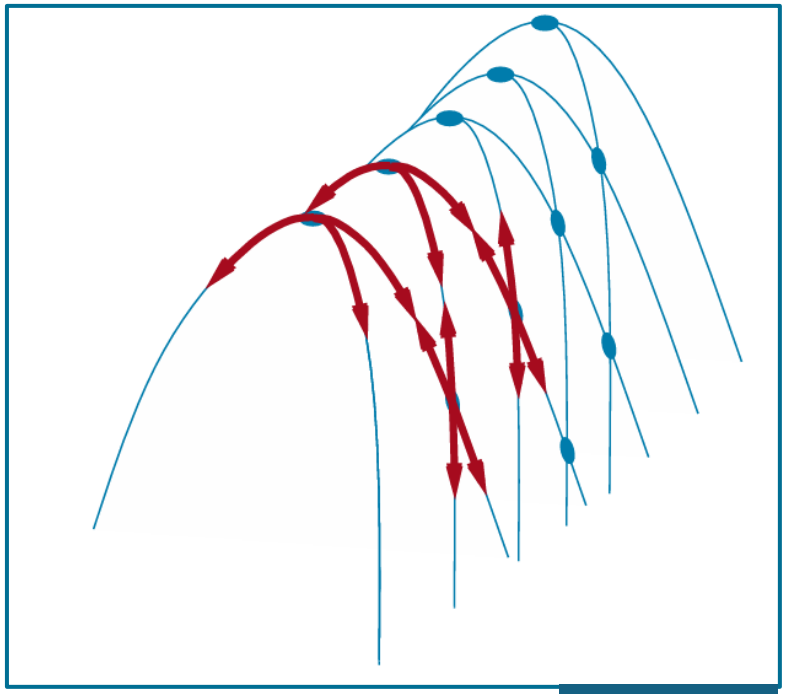
A Heuristic for the n-Queens Problem

- Goal: n queens on the board with no conflicts - no queen attacking another.
- States: n queens on the board, one per column.
- Actions: move a queen within its column.
- Heuristic value function: the number of conflicts.
- The three boards show h falling from 5 to 2 to 0 - two moves, and the problem is solved.

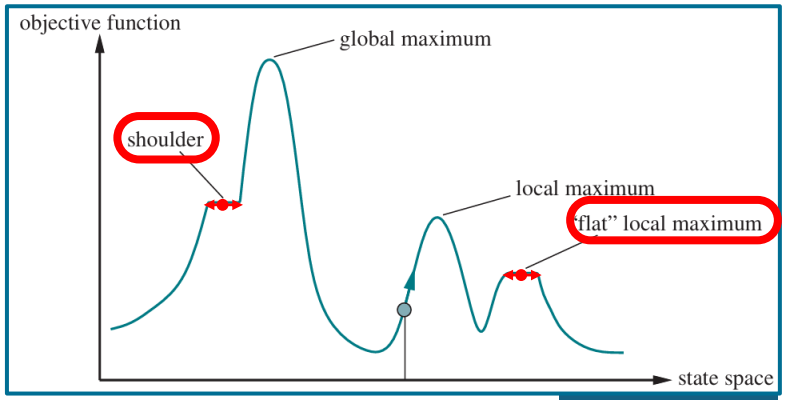


Local Optima: Maxima, Ridges and Plateaus

- **Local maximum**: higher than its neighbors but lower than the global maximum
- **Ridge**: a sequence of local maxima; a greedy algorithm has trouble crossing one
- **Plateau**: a flat region: either a flat local maximum with no uphill exit, or a shoulder from which progress is still possible
- Is hill climbing complete or optimal?
  - ➢ *No, it can get stuck in any of these.*



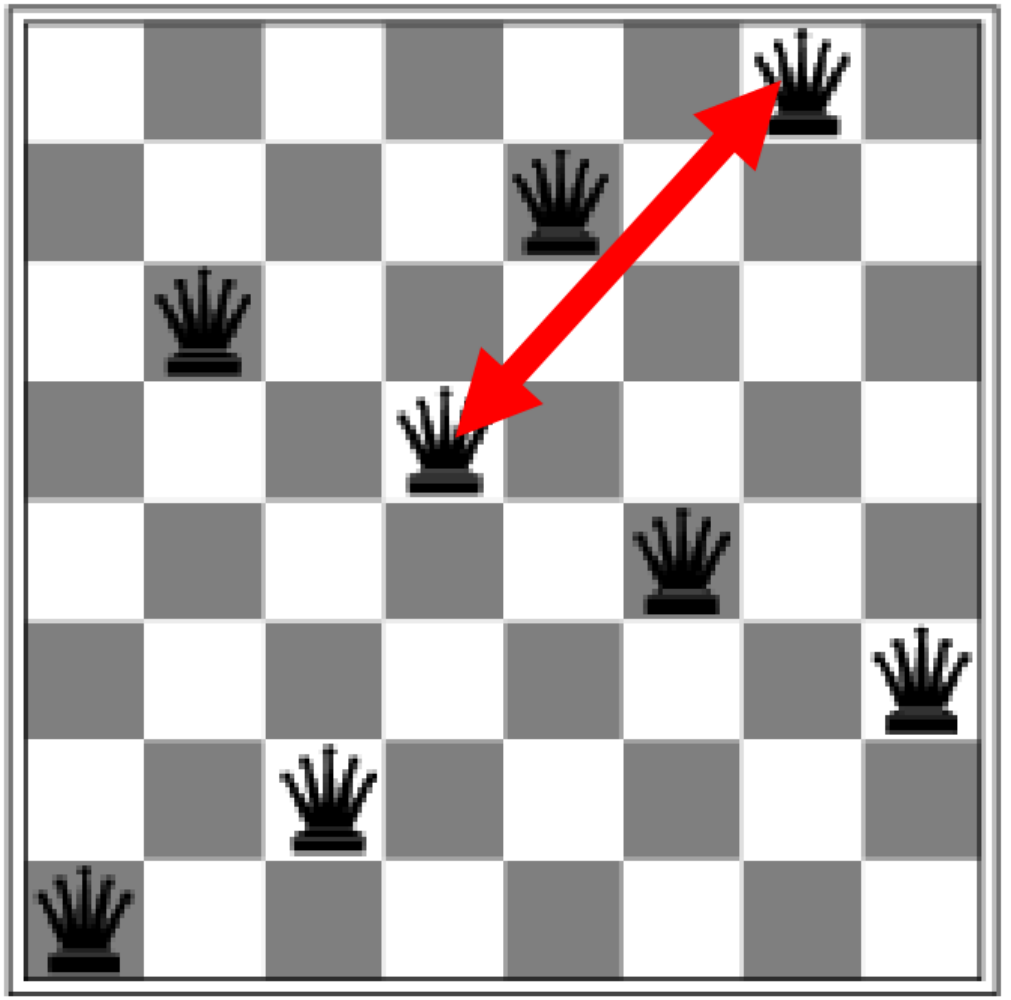
Ridge



Plateau

Local Optima: An Example, and Random Restart

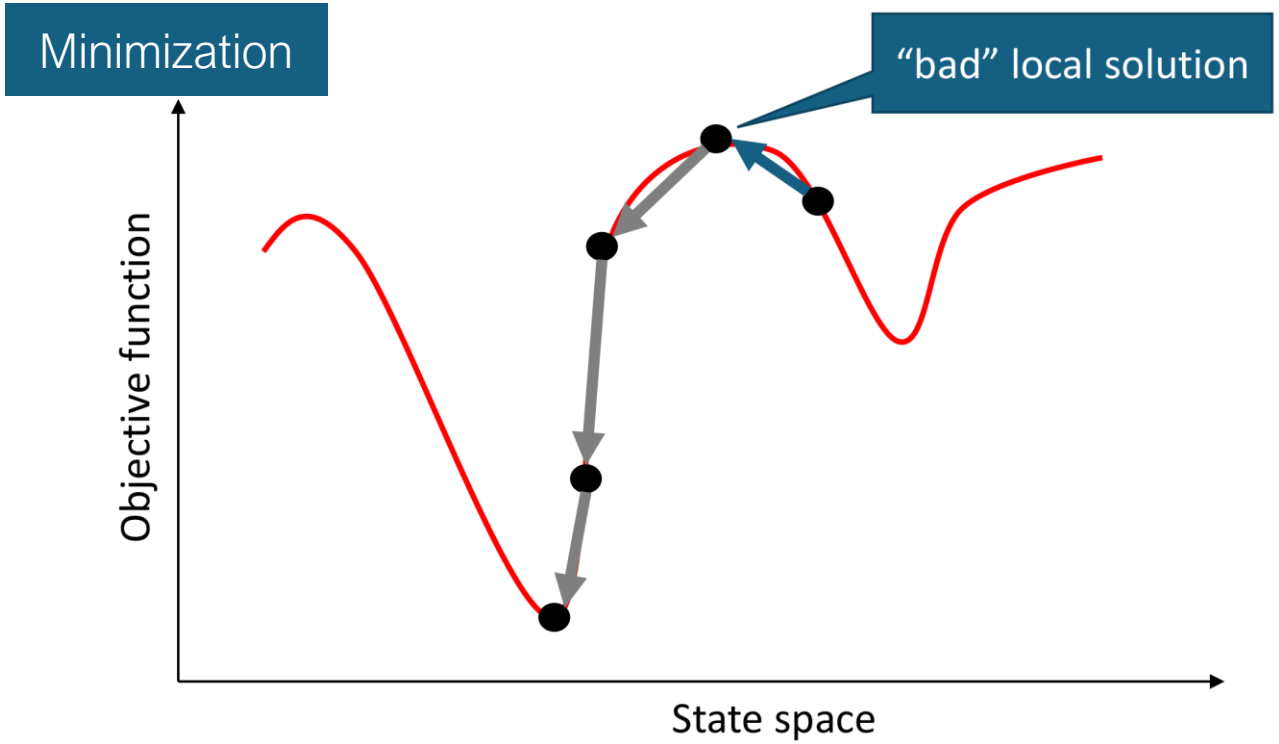
- A local optimum in the 8-queens problem: no queen can be moved within its own column to improve the objective function, yet one attacking pair remains
- **Random-restart hill climbing**: run hill climbing from several random initial states and return the best result found
- That strategy works with any stochastic algorithm, not just this one



$h = 1$

Simulated Annealing: The Idea

- Accept a ***"bad move"*** with some probability, and lower that probability over time, so the search can climb out of a local minimum
- The name comes from annealing: the controlled cooling of glass or metal. Lowering the temperature means lowering the probability of accepting a bad move





Annealing

The Simulated Annealing Algorithm

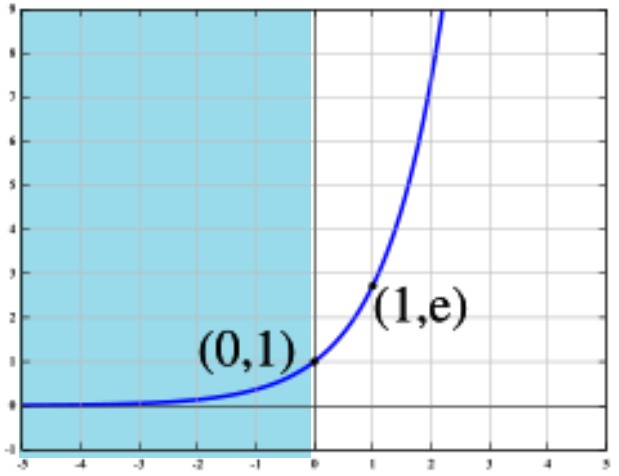
- It is ***first-choice stochastic hill climbing***, plus a rule that accepts some bad moves and accepts them less often as the search goes on.
- A negative change means the solution got better; such a move is always taken
- A bad move is accepted with probability $\exp(-\Delta E \,/\, T)$, where ΔE is how much worse the move is, and the annealing schedule lowers the temperature T over time.

Maximization

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
    current ← problem.INITIAL
    for t = 1 to ∞ do
        T ← schedule(t)
        if T = 0 then return current
        next ← a randomly selected successor of current
        ΔE ← VALUE(current) – VALUE(next)
        if ΔE ≤ 0 then current ← next
        else current ← next only with probability e^(−ΔE/T)
```

Next is better, just move

Next is worse, bad move with a probability



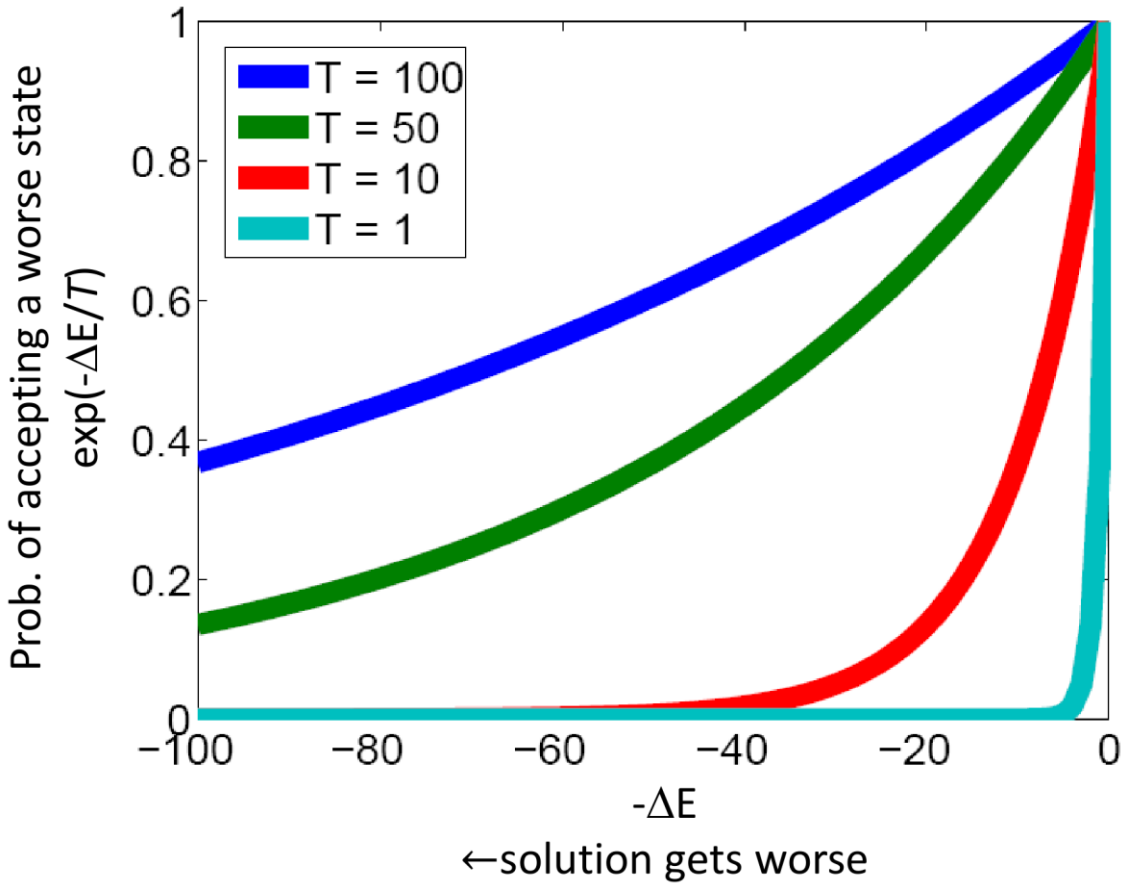
Boltzmann distribution

$p_i \propto e^{-\varepsilon_i/(kT)}$

For minimization, take $VALUE(next) - VALUE(current)$ instead

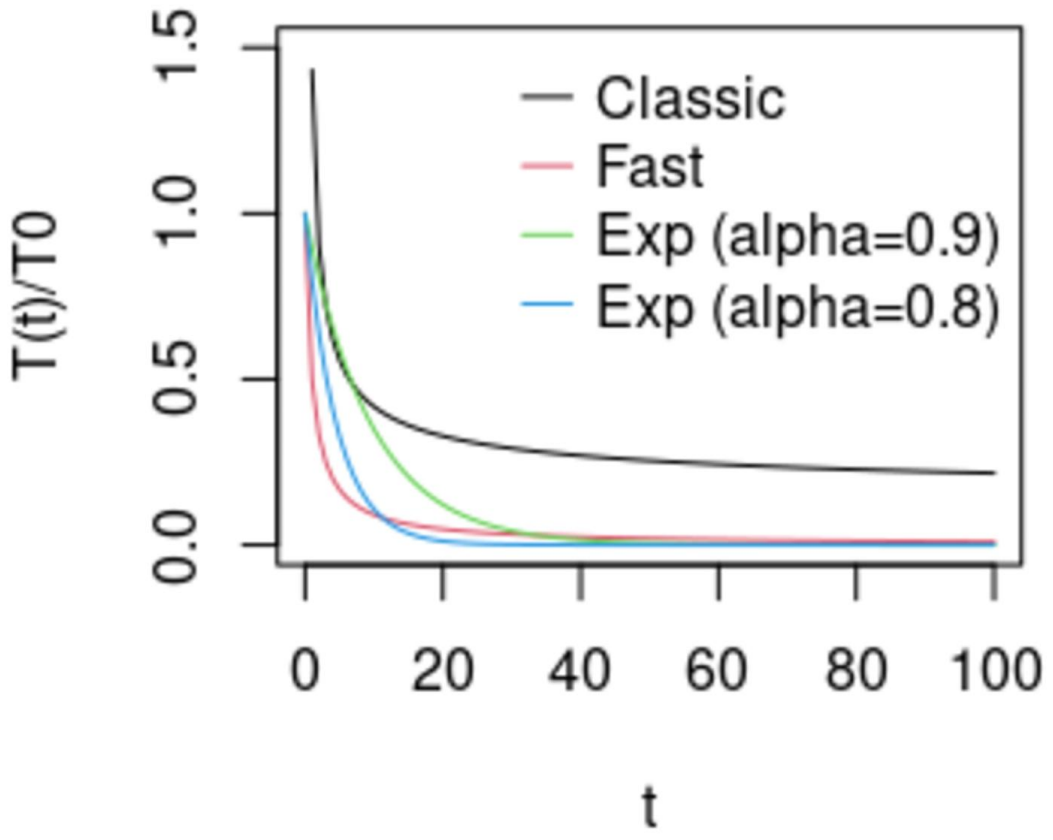
The Effect of Temperature

- The probability of accepting a worse state is $\exp(-\Delta E \,/\, T)$
- At a high temperature the curve is nearly flat: almost anything is accepted
- At a low temperature only a very small loss is accepted, so the search settles



Cooling Schedules

- Classic simulated annealing: $T(t) = \frac{T_0}{\log(1+t)}$
- Exponential cooling (Kirkpatrick, Gelatt and Vecchi, 1983):
  $$T(t) = T_0 * a^t \text{ for } 0.8 < a < 1$$
- Fast simulated annealing (Szu and Hartley, 1987):
  $$T(t) = \frac{T_0}{1+t}$$
- Choose T_0 so that at t = 0 almost any move is accepted; stop once T falls below a small constant.
- The best schedule is normally found by trial and error
  - AI researchers do always 😅.



Properties of Simulated Annealing

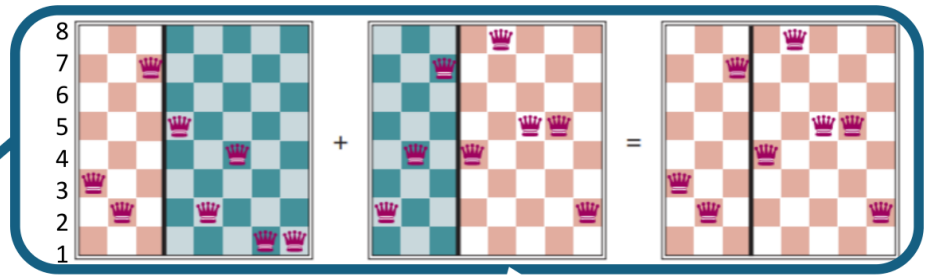
- If the temperature falls "***slowly enough***", simulated annealing finds the global optimum.
  - At a high temperature, almost every move is accepted, and the search wanders the whole space freely
  - As the temperature falls, a large loss is rarely accepted but a small one still is — there is room left to climb out of a local optimum
  - At a very low temperature, a worse solution is effectively never accepted, and the search stays near the optimum it has
- But…,
  - In practice, this guarantee usually takes **far too long**.
  - Both the cooling schedule and the local move have to be found **experimentally**.

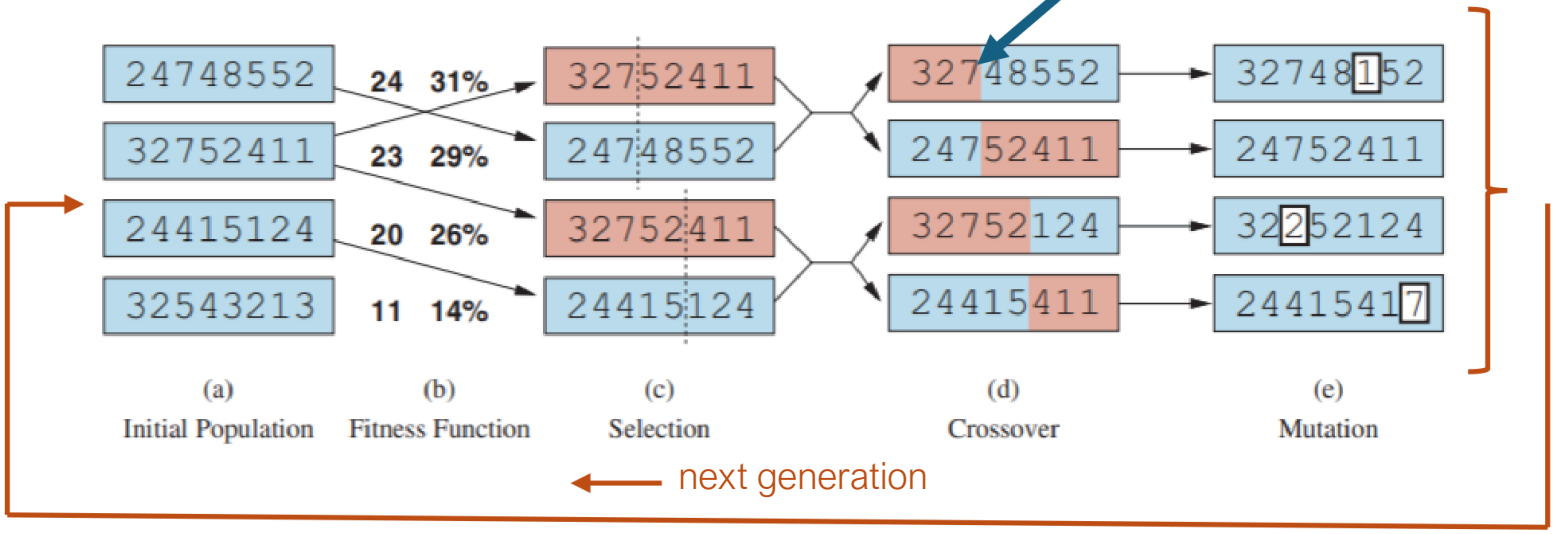
Local Beam Search

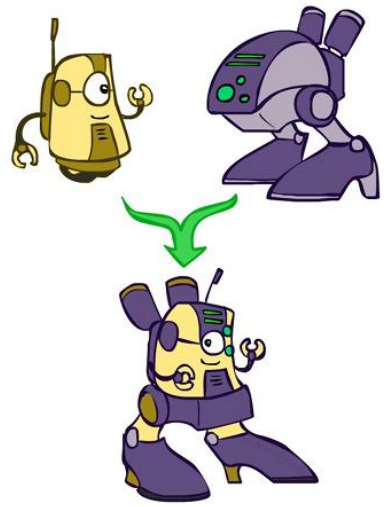
- The **search** that follows ***k* states** at once rather than one, starting from k randomly chosen states
  - Unlike random restart it does not follow a single path, it keeps k states, **the beam width**, in parallel.
- How it runs:
  - choose k states at random from the initial set
  - generate ALL successors from k states
  - keep **the best** k of all those successors and repeat
  - stop at a goal state or when a termination condition is met
- Properties
  - More efficient than random restart (=single-path)
    : information is shared across the k states (=communicate)
  - Stronger than hill climbing, because several directions are explored at once
  - k too small: it collapses into a local optimum.
  - k too large: it approaches exhaustive search and becomes inefficient.

Genetic Algorithms

- An algorithm for ***population-based*** optimization, using mechanisms borrowed ***from biological evolution***
  - Reproduction: select at random, with probability weighted by a fitness function $(1 - h(n))$
  - Crossover: recombine two parents at a random point
  - Mutation: change a random position
  - Repeat for many generations.
- In the 8-queens example



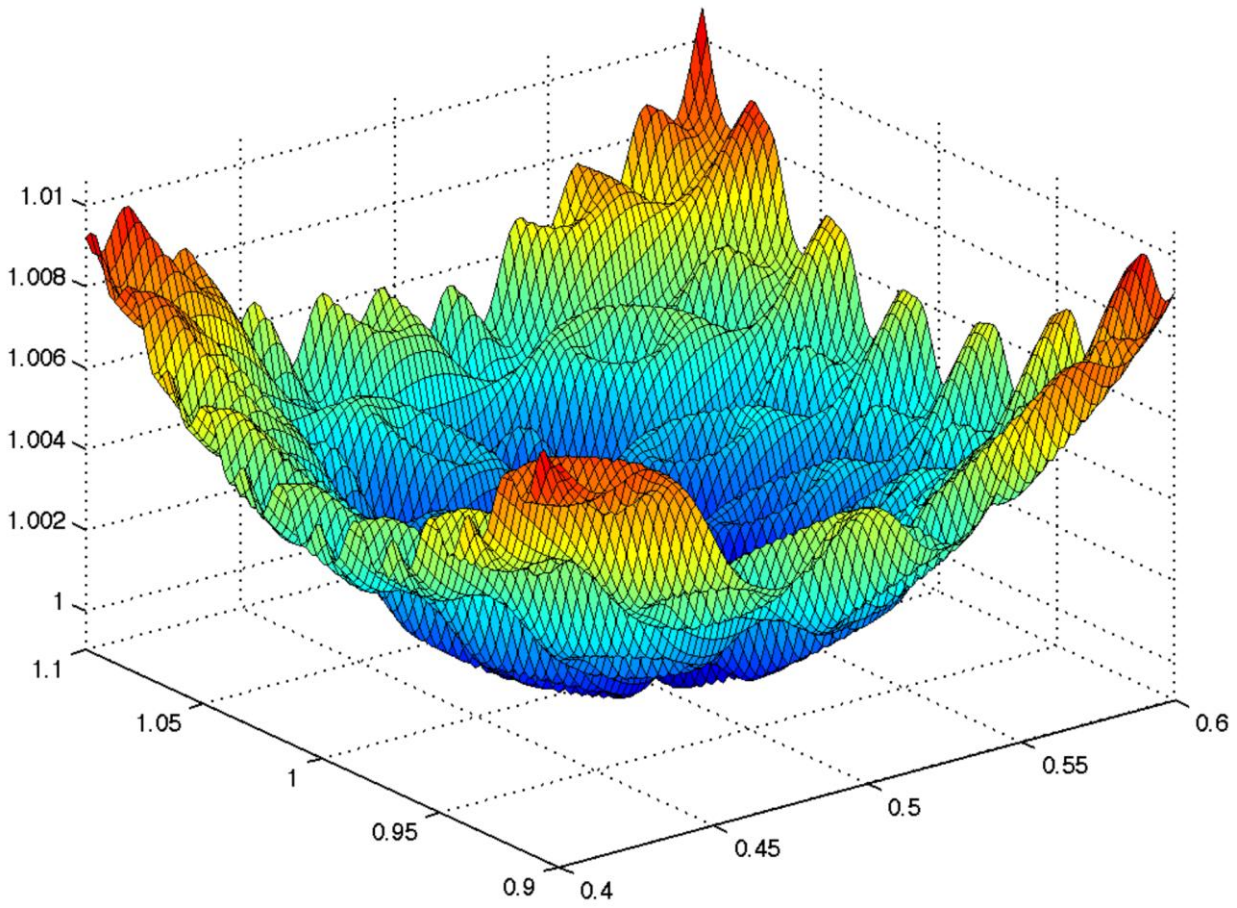




Local Search in Continuous Spaces

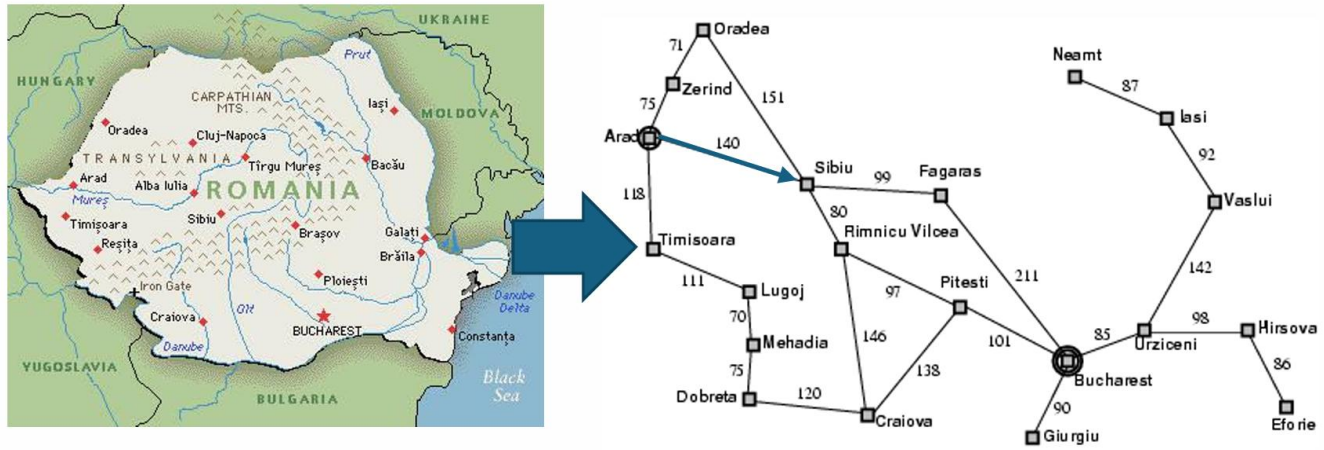
- Most real optimization problems live in a continuous space: the state is a vector of real numbers, not one of a finite set.

- There is no finite list of successors to enumerate, so hill climbing as stated does not apply.

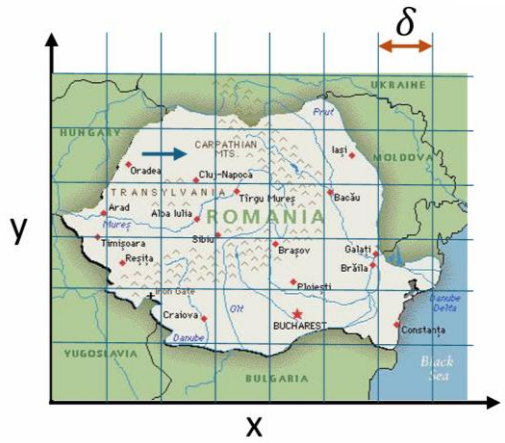


Method 1 - Discretization

- Use atomic states and build the transition function as a graph.



- Lay a grid of spacing delta $\boldsymbol{\delta}$ over the space
  ; the neighbors of a point are the adjacent grid cells

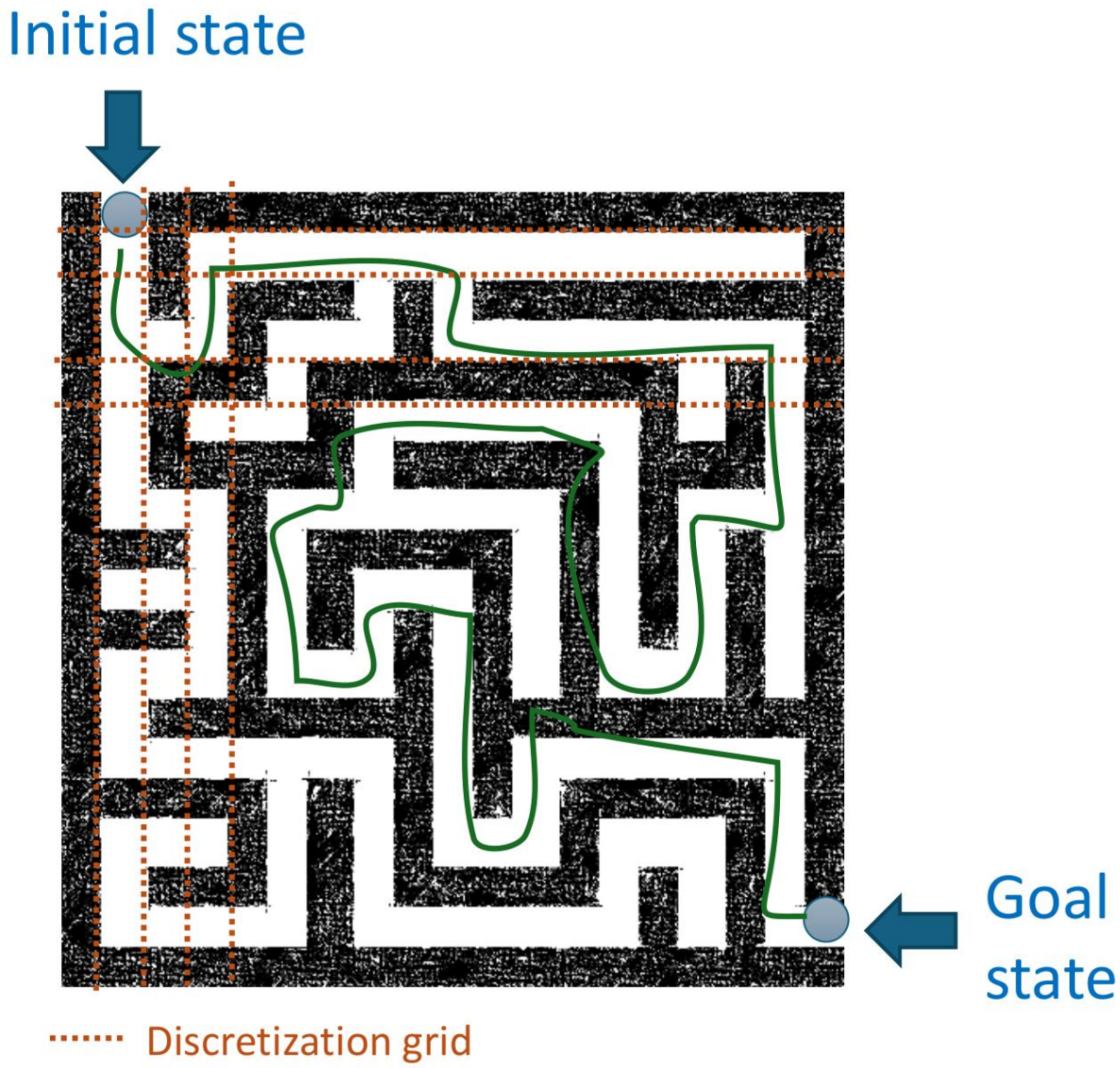


- In practice a much finer grid than the one drawn here is usually needed

Discretisation: An Example

- How was this space discretized? The dotted lines are the grid, and the path is a sequence of grid cells

- The maze is actually continuous, but once the grid is applied the problem is an ordinary graph search from the initial state to the goal state.



Three Ways to Handle a Continuous Space

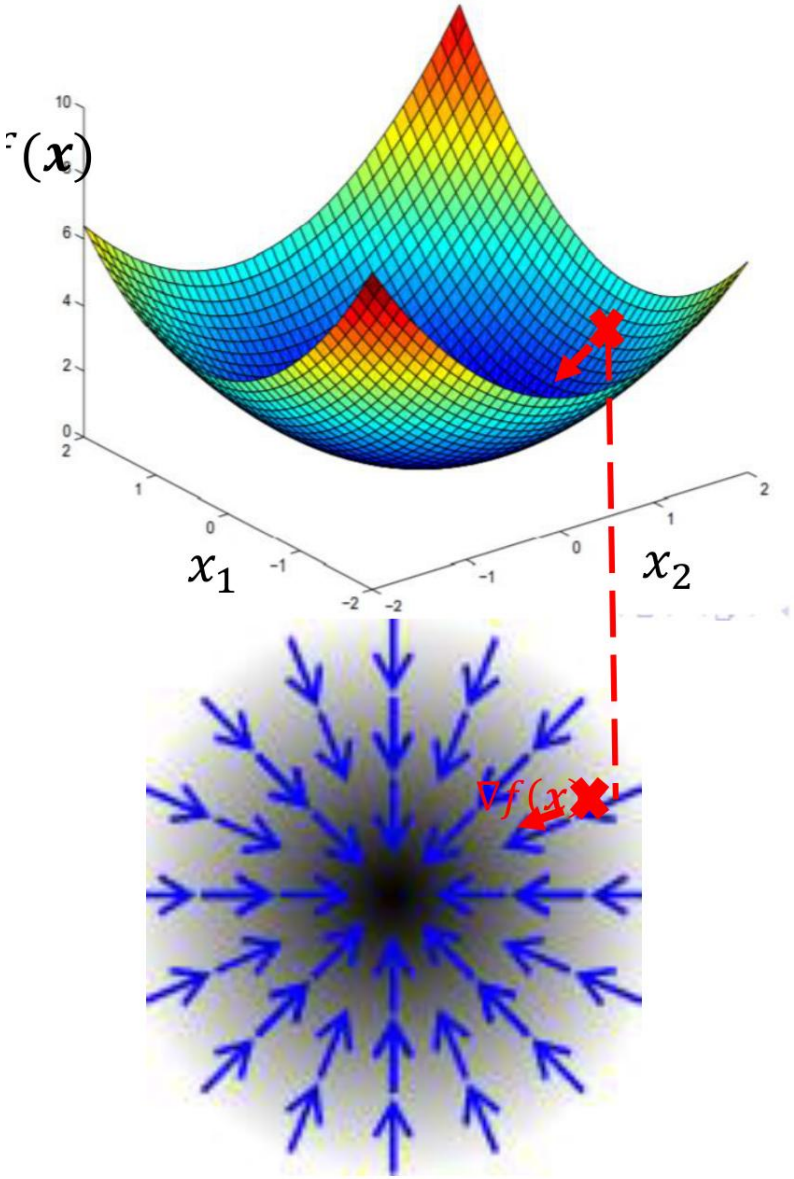
1. Discretize it!
   - define a grid with increment delta δ and use any of the discrete algorithms.

2. Choose between random perturbations to the state
   - hill climbing with simulated annealing.

3. Use continuous, analytic methods
   - compute gradients and so on, to extrapolate the local curvature.

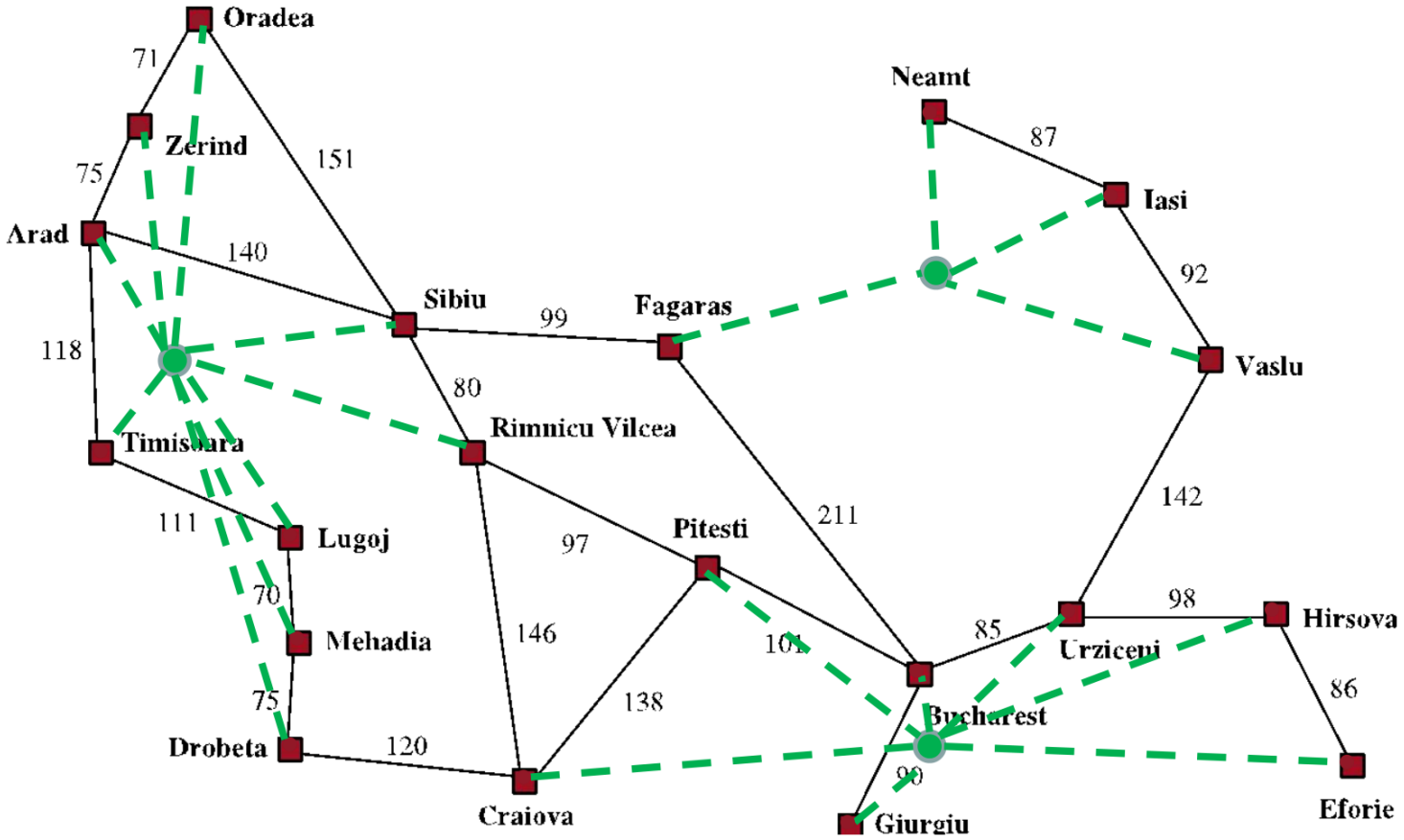
Gradient Descent and Newton-Raphson

- State: a vector $x = (x_1, x_2, \ldots, x_k)$.
- The state space is **infinite**.
- The objective function: $\min f(x)$
- The gradient at x: $\nabla f(\mathbf{x}) = \left(\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \ldots, \frac{\partial f(\mathbf{x})}{\partial x_k}\right)$
- Gradient descent iterates with a step size α:
$$x \leftarrow x - \alpha * \nabla f(x)$$
- **Newton-Raphson** uses the inverse Hessian as an optimal step size: $x \leftarrow x - H_f^{-1} \nabla f(x)$
- Note: In a non-convex space, both still get trapped in a local optimum: **simulated annealing**, **momentum** or a similar device is used to escape



Example: Siting Airports in Romania

- Place 3 airports to minimize the sum of squared distances from each city to its nearest airport.



- The state is the airport locations:
  $\mathbf{x} = (x_1, y_1), (x_2, y_2), (x_3, y_3)$
- City locations: (x_c, y_c)
- C_a is the set of cities closest to airport a.
- Objective: minimize

$$f(\mathbf{x}) = \sum_{a} \sum_{c \in C_a} [(x_a - x_c)^2 + (y_a - y_c)^2]$$

Many Machine Learning Algorithms Are Local Search

- Many configuration and optimization problems can be formulated as local search.
- General families of algorithms:
  - Hill climbing, and continuous optimization.
  - Simulated annealing, and other stochastic methods.
  - Local beam search: multiple interacting searches.
  - Genetic algorithms: break states apart and recombine them.
- Many machine learning algorithms are local searches.

Introduction to Artificial Intelligence

Chapter 4 - Part II: Nondeterminism

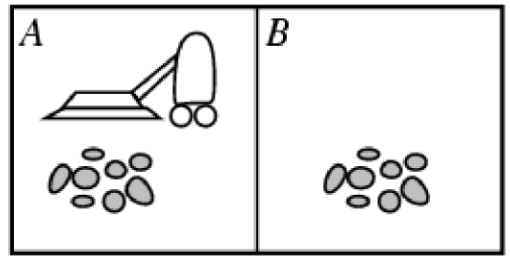
Namyup Kim

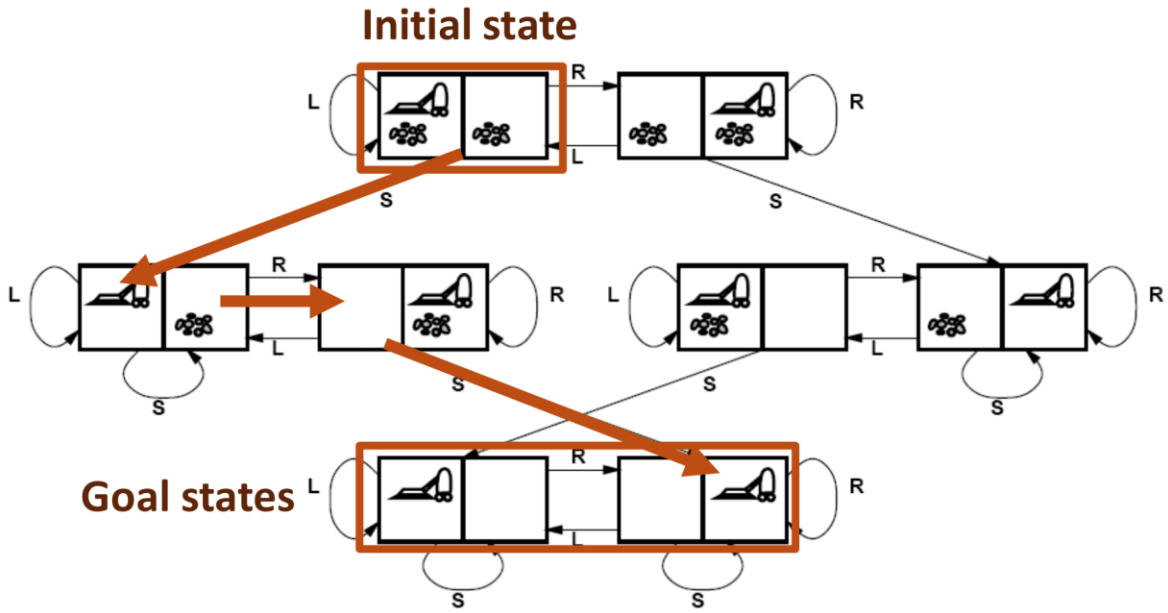
Soongsil University



Recap: Solving a Search Problem under Certainty

- **No uncertainty**: the environment is fully observable, so the agent knows the state, and it is deterministic, so $Result(s, a)$ is a single state.
- The state space describes the environment and the agent completely
- Method: use tree search at planning time to produce a sequence of actions, then execute that plan as it stands.





Where Uncertainty Comes From, and What It Costs

- Two sources:
  1. The **agent**'s actions are nondeterministic: it cannot determine the state the environment will be in after acting
  2. The **environment** is only partially observable: the agent is uncertain about the state it is in now
- Two consequences:
  - A fixed and precomputed plan (a sequence of actions) does not work against a stochastic transition function. What is needed instead is **a conditional plan (a strategy or a policy)** that depends on the percepts.
  - The agent must track every state it could be in. That set of states is called ***a belief state***.

1. Nondeterministic Actions: The Definition

- If the agent's actions in the environment are **nondeterministic** , the transition model has to describe that **uncertainty**

- So, $Results(s, a)$ returns a set of states rather than one state.

- For example,

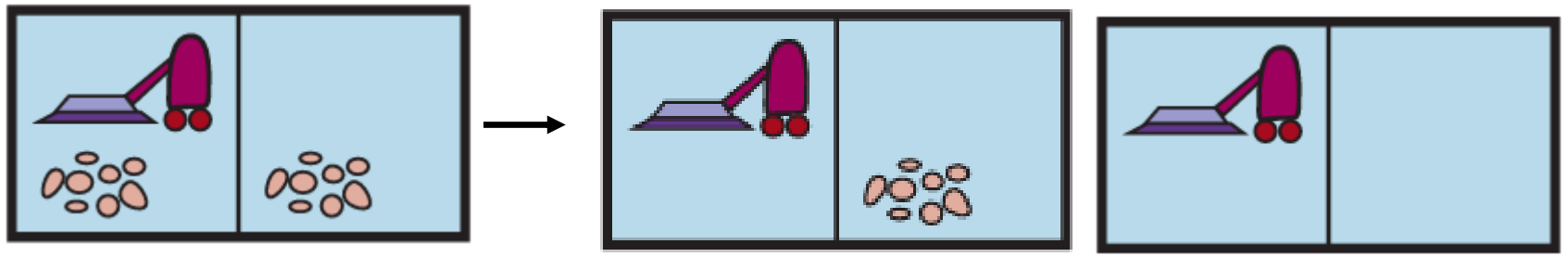
$$Results(s_1, a) = \{s_2, s_4, s_5\}$$

  - Action a in state $s1$ can lead to any one of three states.

- That set $\{s_2, s_4, s_5\}$ is a belief state: after acting, the agent knows only that it is in one of them.

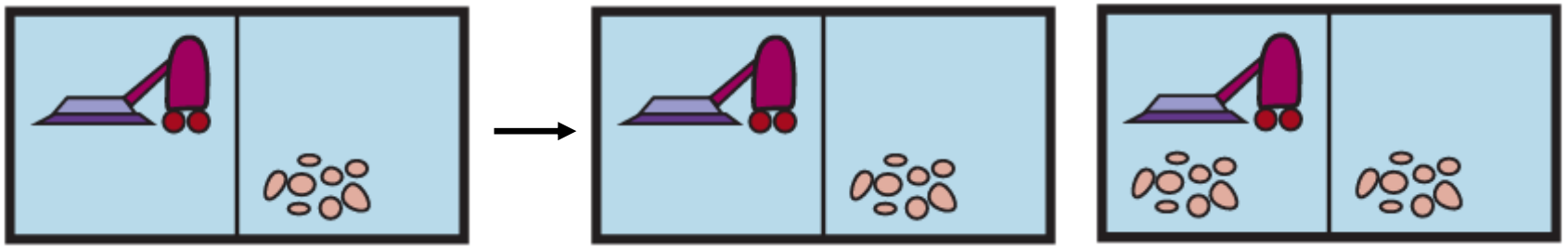
Example 1: The Erratic Vacuum World

- The ordinary deterministic vacuum world, *except* that the **powerful** but **erratic "`Suck`"** action makes it nondeterministic
  - (a) On a dirty square: it cleans that square, and sometimes cleans the adjacent square too



$$Results(s_1, a) = \{s_5, s_7\}$$

  - (b) On a clean square: it sometimes deposits dirt on that square



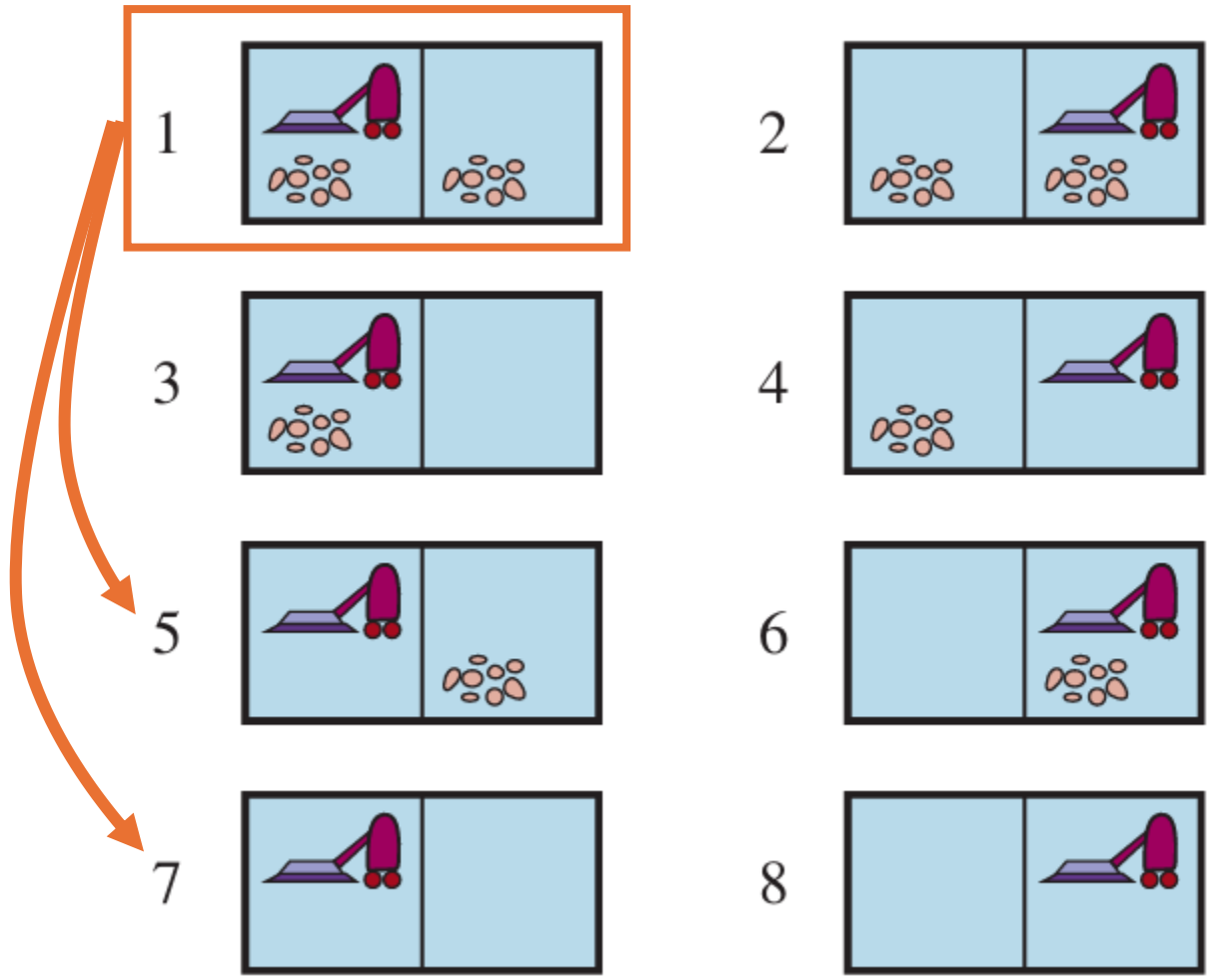
The Erratic Vacuum World

- From state 1, the **“Suck”** action can lead to two different states , so a ***conditional plan*** is needed.

- The solution in a nondeterministic environment.
  - NOT a sequence of actions
  - BUT a tree containing **`if-then-else`** steps.

```
[Suck, if State = 5 then [Right, Suck] else []]
```
Determined initial action

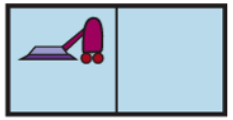
State = 7



- *The solution of The ordinary deterministic vacuum world was* ***`[Suck, Right, Suck]`***

AND-OR Search Trees

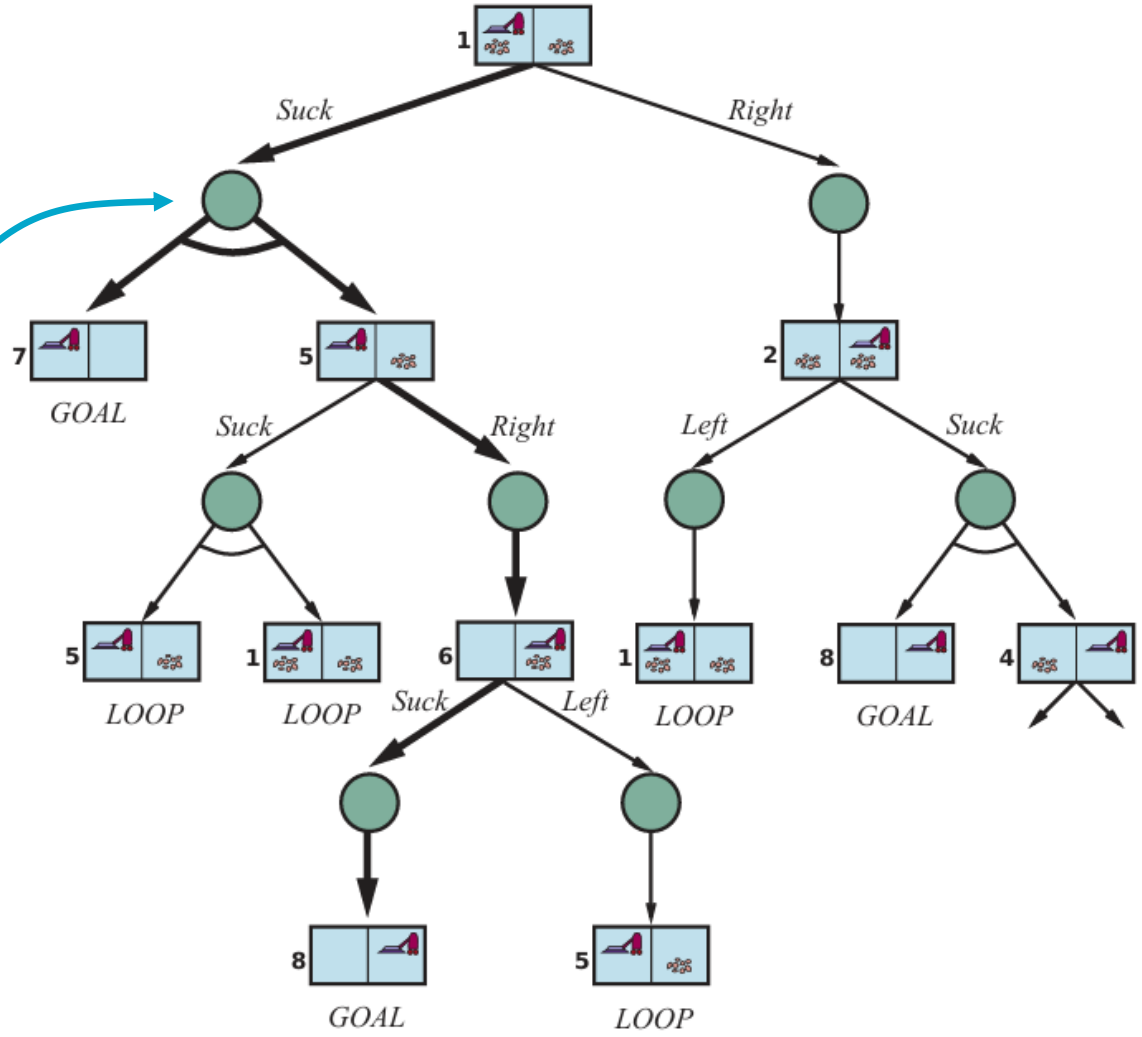
- OR node: the **agent** chooses one action among several.



- AND node: the **environment** chooses, and **every possible outcome** must be handled.

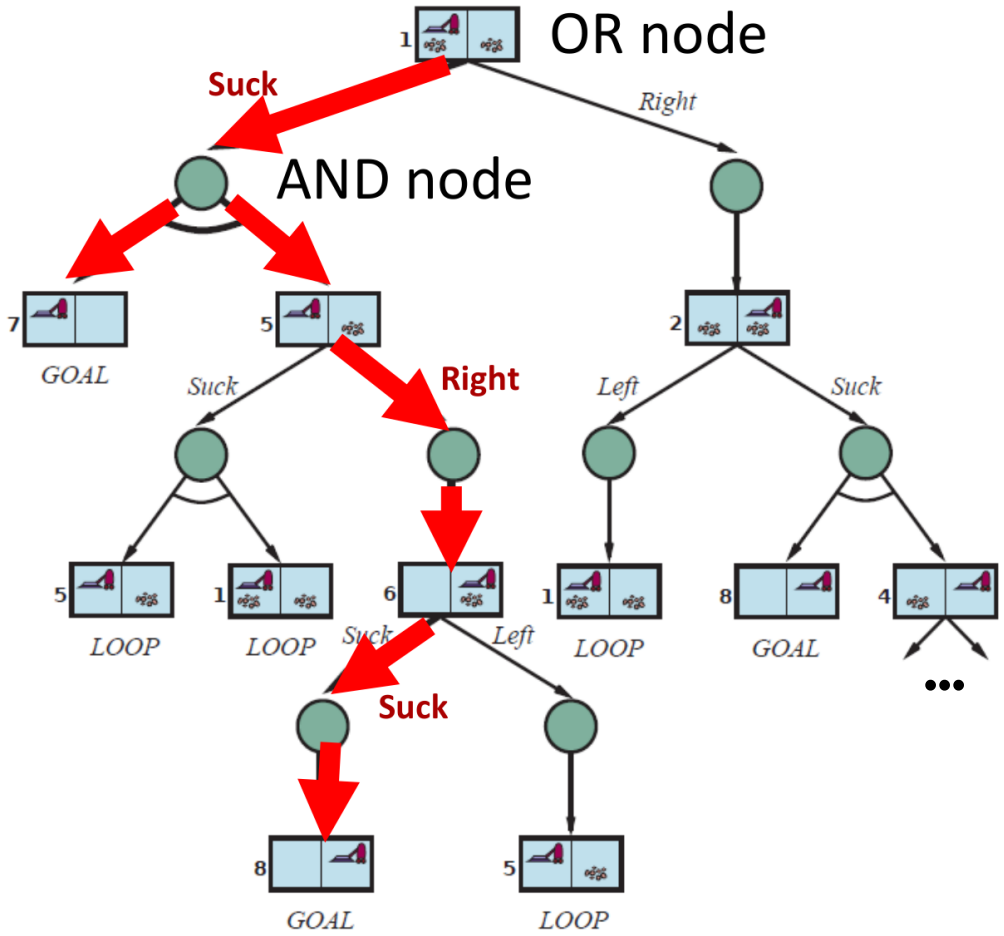


- Because **"Suck" in state 1** leads to the belief state {5, 7}, the agent needs a plan for state 5 AND a plan for state 7.



Searching the AND-OR Tree

- Goal: choose one action at every **OR** node, cover every outcome at every **AND** node, and end with a subtree whose leaves are all goal states
- Descend depth-first:
  - At an OR node try one action at a time;
  - At an AND node take every outcome and recurse
  - if a state already appears on the current path, return failure
  - abandon any subtree whose leaves are not all goals;
  - stop at the first complete subtree found



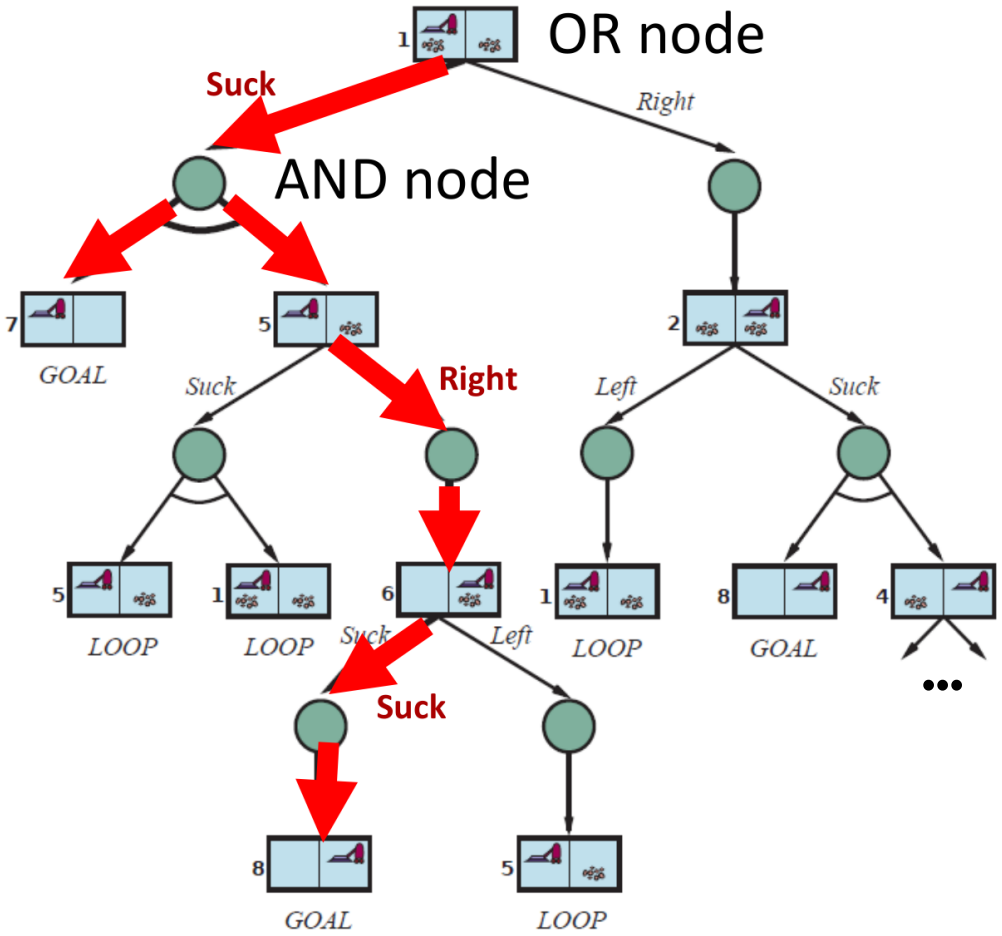
Conditional Plan:

```
[Suck, if State = 5 then [Right, Suck] else []]
```

Recursive AND-OR Tree Search (DFS)

- DFS: NOT OPTIMAL (Find path and just `return`)

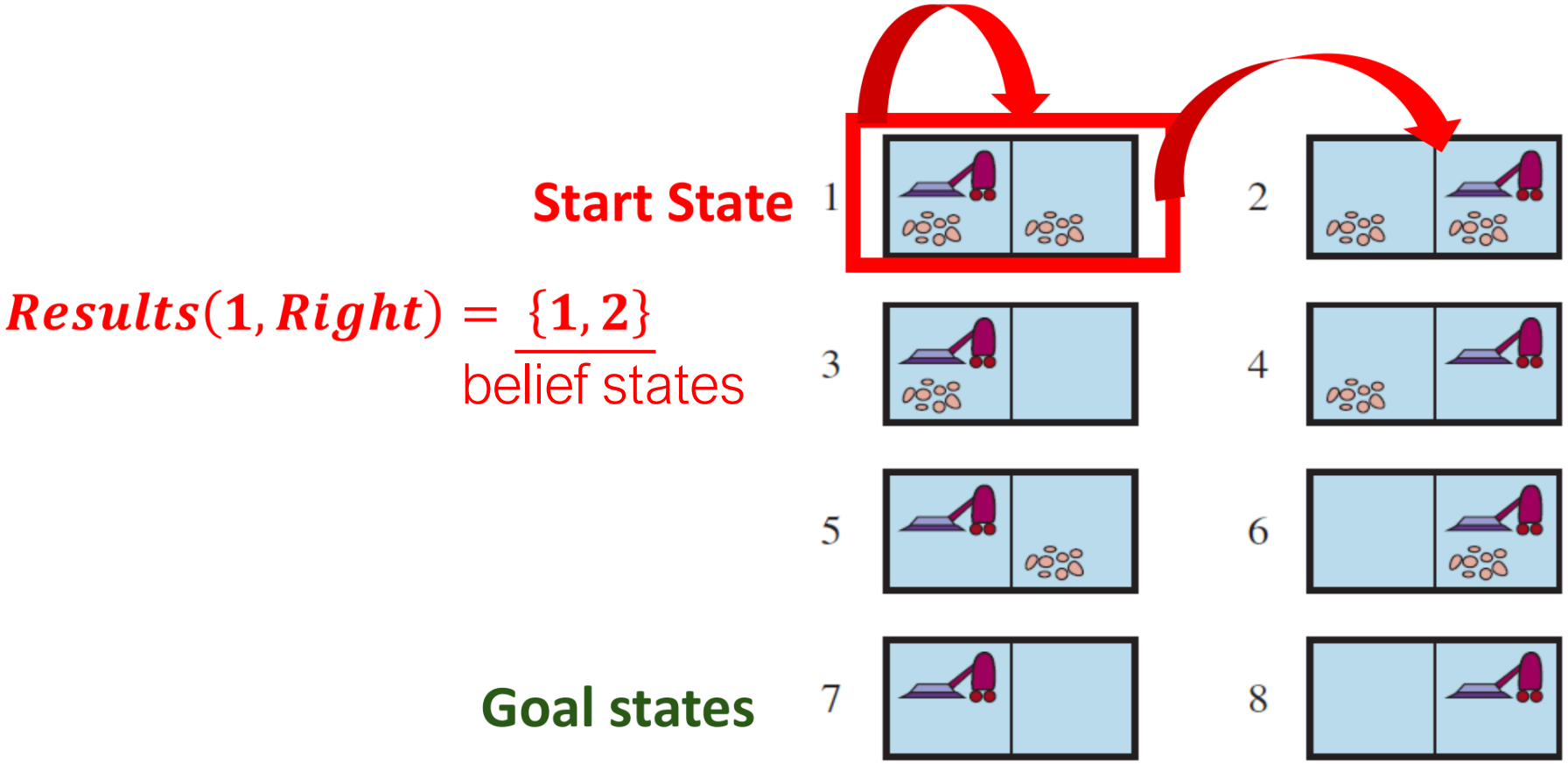
OR-SEARCH(1, [])
- ➔ "Suck" -> RESULTS = {5, 7}
  - ➔ AND-SEARCH({5,7}, [1])
    - ➔ OR-SEARCH(5, [1])
      - ➔ "Suck" -> RESULTS = {5, 1}
        - ➔ AND-SEARCH({5,1},[1, 5])
          - ➔ OR-SEARCH(5, [1, 5]) IS-CYCLE -> `return fail`
          - ➔ OR-SEARCH(1, [1, 5]) IS-CYCLE -> `return fail`
      - ➔ "Right" -> RESULTS = {6}
        - ➔ AND-SEARCH({6}, [1, 5])
          - ➔ OR-SEARCH(6, [1, 5])
            - ➔ "Suck" -> RESULTS = {8}
              - ➔ AND-SEARCH({8}, [1, 5, 6])
                - ➔ OR-SERCH(8, [1, 5, 6]) GOAL `return ["Right", "Suck"]`
            - ➔ "Left" -> RESULTS = {5}
              - ➔ AND-SEARCH({5}, [1, 5, 6])
                - ➔ OR-SERCH(8, [1, 5, 6]) IS-CYCLE -> `return fail`
    - ➔ OR-SEARCH(7, [1]) GOAL `return []`
    - ➔ `return [if State = 5 then [Right, Suck] else []] (n=2)`
  - ➔ `return ["Suck", if State = 5 then [Right, Suck] else []]`



Conditional Plan:
`[Suck, if State = 5 then [Right, Suck] else []]`

Example 2: The Slippery Vacuum World

- The ordinary deterministic vacuum world, *except* that a movement action (Right or Left) sometimes fails and the agent stays put.
- So, from state 1, **"`Right`"** gives the belief state $\{1, 2\}$: $Results(1, Right) = \{1, 2\}$



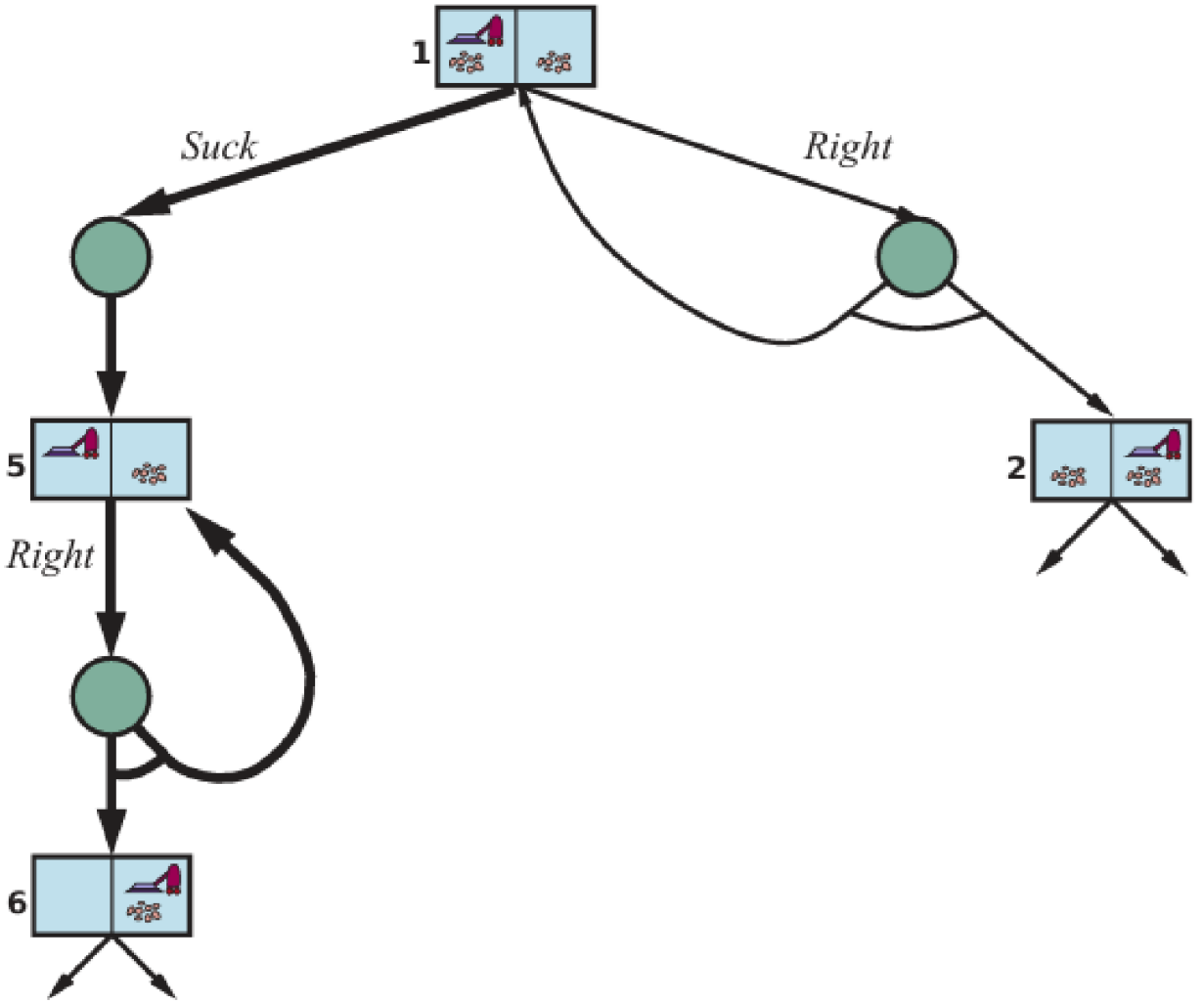
Cyclic Solutions

- There is no longer an acyclic solution from state 1, and **AND-OR search returns failure (it has IS-CYCLE check)**

- But a cyclic solution does exist: keep trying Right until the move succeeds

  $[Suck, while\ State\ =\ 5\ do\ Right, Suck]$

- ***The plan is now a loop, not just a branch*** which is why a plan is better thought of as a program than as a list.



Introduction to Artificial Intelligence

Chapter 4 - Part III: Partial Observability

Namyup Kim

Soongsil University



1. Sensorless (Conformant) Problems

- A sensorless (or conformant) problem: the agent has no percepts at all, and gets no information from the environment

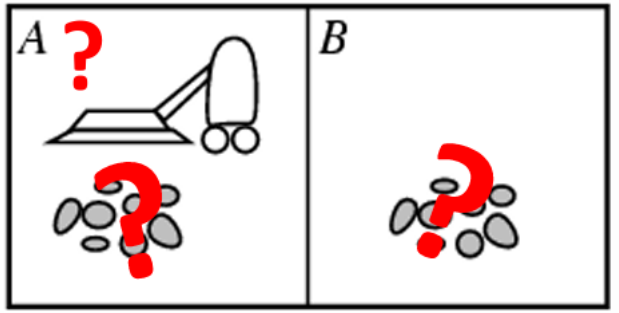
- Why is that useful?
  - A doctor prescribes just a broad-spectrum antibiotic instead of running a slow blood test to identify a more specific one - saving time and cost

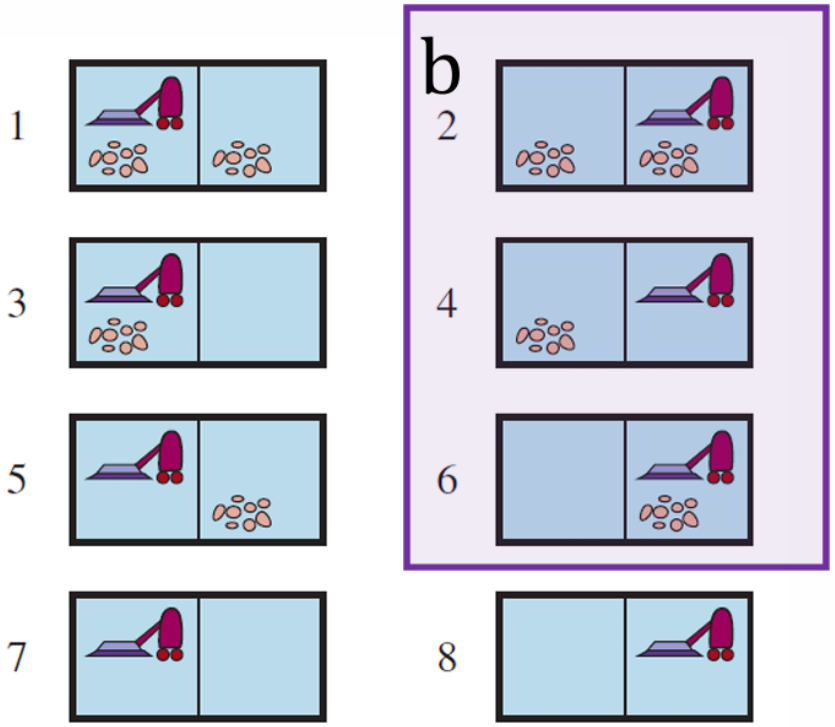


- The basic idea: find a plan that works reasonably well whatever state you are in, then execute it blind

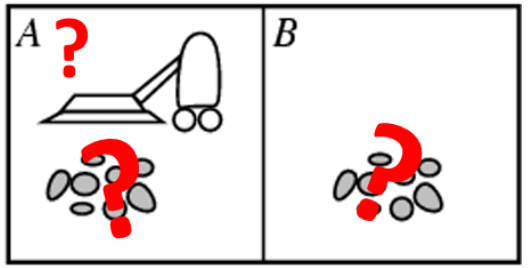
Belief States

- The agent does not know exactly which state it is in.
- But it does know it is in one of several possible states.
- That set is the **belief state**: for example, $b = \{s2, s4, s6\}$





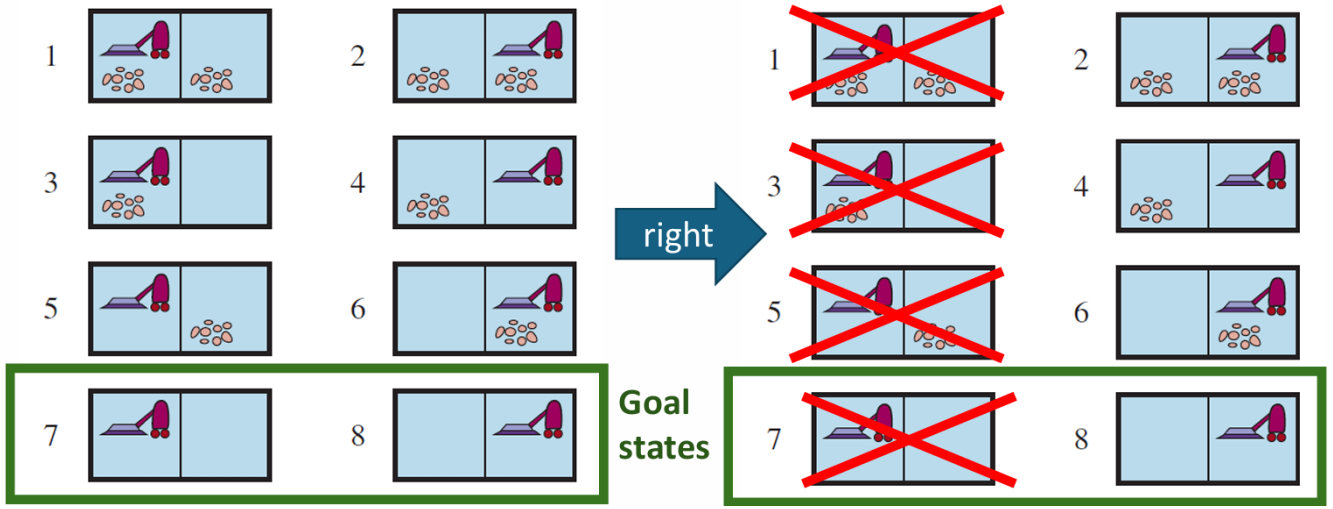
An Action Can Narrow the Belief State



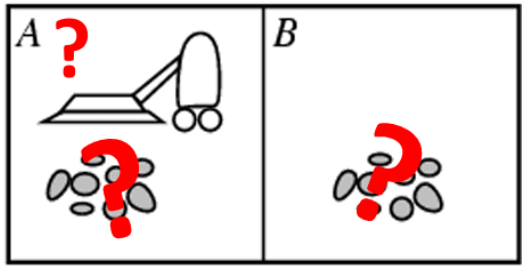
- The initial belief state is everything:

$$\{1, 2, 3, 4, 5, 6, 7, 8\}$$

- **An action can reduce the number of possible states.**
- Example: the deterministic but unobservable vacuum world
  - The agent knows neither its own position nor where the dirt is
  - After `Right`, every state in which the agent was on the left has moved right, so half the possibilities collapse onto the other half.



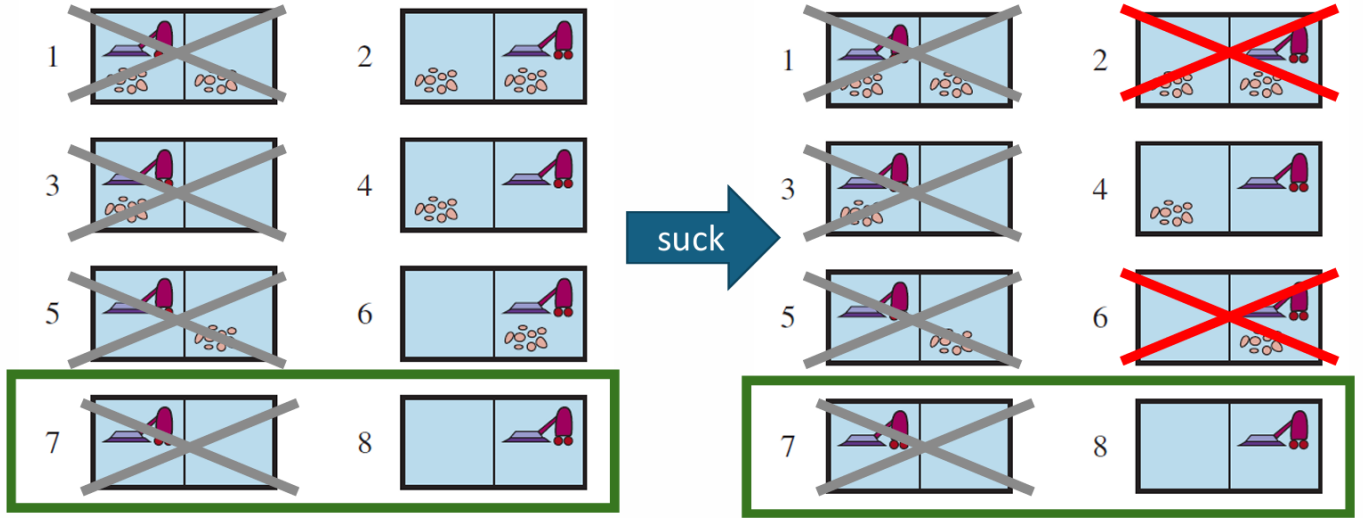
An Action Can Narrow the Belief State



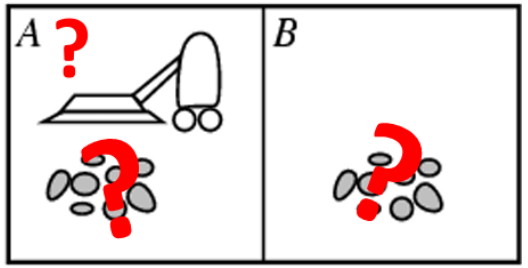
- The initial belief state is everything:

$$\{1, 2, 3, 4, 5, 6, 7, 8\}$$

- **An action can reduce the number of possible states.**
- Example: the deterministic but unobservable vacuum world
  - After `Right`, every state in which the agent was on the left has moved right, so half the possibilities collapse onto the other half.
  - After `Suck`, the square the agent is on is clean in every remaining possibility, so the belief state shrinks again.



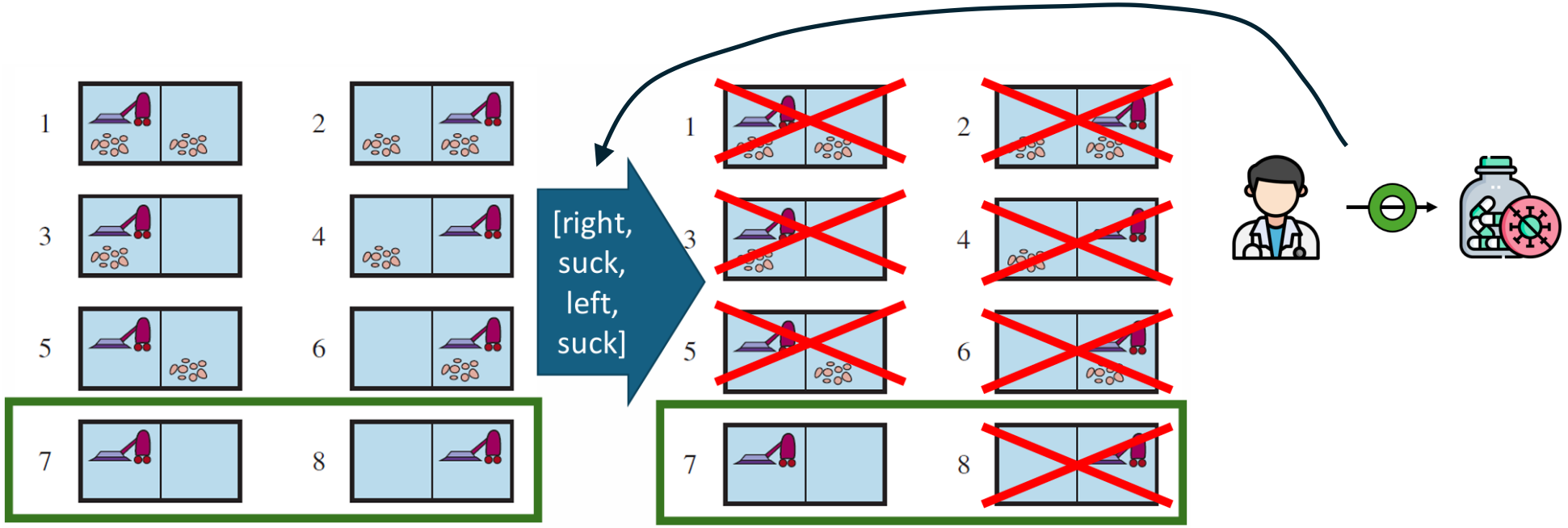
An Action Can Narrow the Belief State



- The initial belief state is everything:

$$\{1, 2, 3, 4, 5, 6, 7, 8\}$$

- Example: the deterministic but unobservable vacuum world
  - The action sequence [`Right, Suck, Left, Suck`] forces the world into goal state 7, whatever state it started in
  - Note that because there are no observations at all, no conditional plan is needed - a plain sequence is enough.

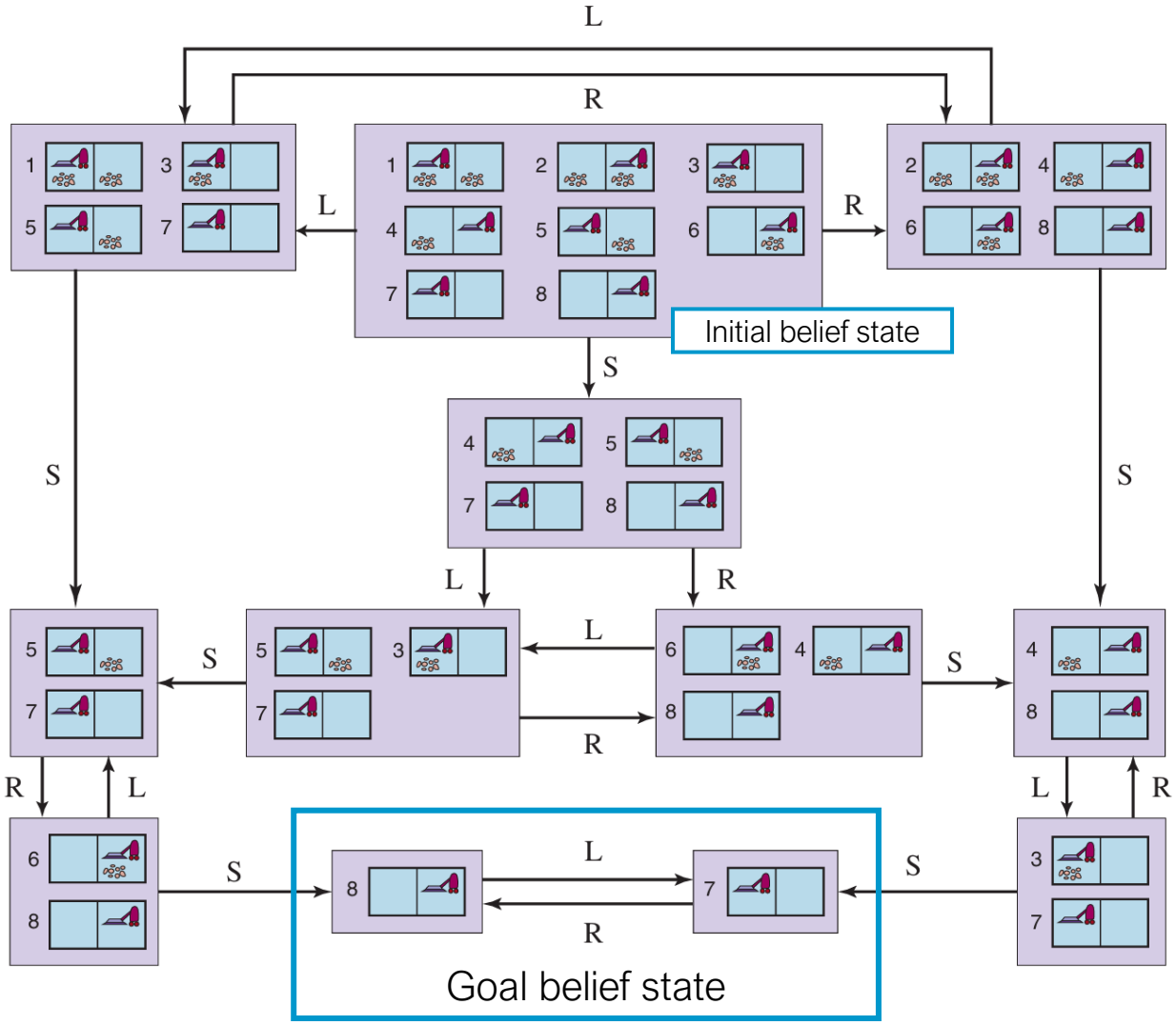


The Reachable Belief-State Space

- With no observations the solution is read off as a sequence:
  [`Right, Suck, Left, Suck`]

- The belief-state space is the power set of the N original states, so its size is 2^N: here $2^8 = 256$

- But, **12 belief states** is actually reachable by acting.



Planning in Belief-State Space

- Formulate it as an ordinary search problem and solve it with **DFS**, **IDS**, **BFS** or **A***:
  - States: Every belief state; there are 2^N of them for N world states.
  - Initial state: Usually the belief state containing every state
  - Actions: $ACTIONS(b) = \bigcup_{s \in b} ACTIONS_P(s)$ where P is the original problem.
  - Transition model: $b' = Results(b, a) = \{ s' : s' = Result(s, a) \; and \; s \; is \; in \; b \}$
  - Goal test: does the belief state contain only goal states?
- Simplifying property: if a belief state has a solution, so does every subset of it, by the same action sequence which prunes the tree.

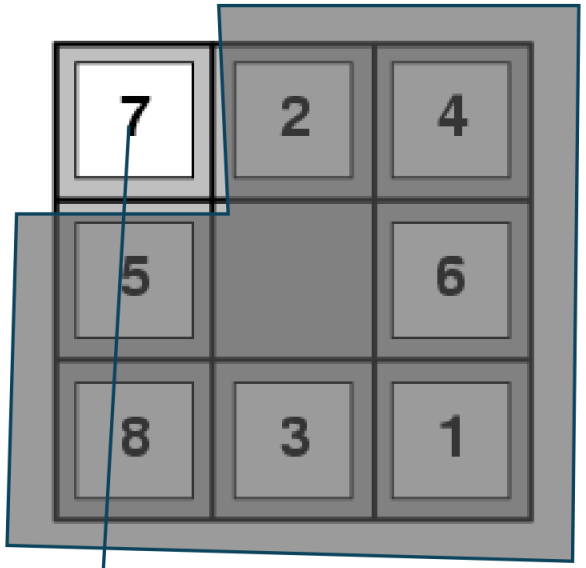
Situation	What follows	Effect on the search
b_1 is already solved by a sequence σ, and a subset $b_2 \subseteq b_1$ comes up later	σ solves b_2 as well	Reuse the plan: no search needed
b_2 is known to be unsolvable, and a superset $b_1 \supseteq b_2$ comes up later	b_1 is unsolvable too	Discard the whole branch
The search reaches b', a superset $b \supseteq b'$ has already reached	b dominates b′	Drop the b′ path - smaller tree

- In practice, the size of the state space 2^N makes this unuseful for larger problems

2. Percepts and Observability

- Many problems cannot be solved efficiently without sensing.
  - Example) The 8-puzzle where the agent must see at least one tile.

- The **percept function** $Percept(s)$ says what the agent observes in state s:
  - Fully observable: $Percept(s) = s$
  - Sensorless: $Percept(s) = None$
  - Partially observable: $Percept(s) = o$
    , an observation that tells the agent something about s.



The difficulty
Many states (different arrangements of the unobserved tiles) produce the same observation

Using Observations: Predict and Update

- Start from the current belief state b: the set of states the agent could be in
- **Predict** with an action: choose a, and use the transition model to get the belief state that results.

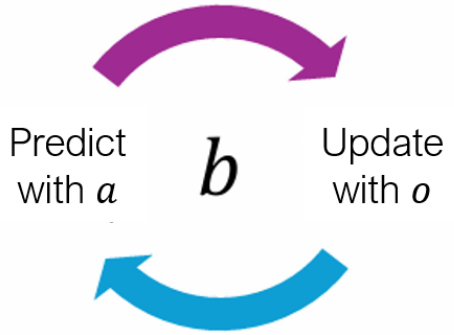
$$\hat{b} = PREDICT(b, a) = RESULT(b, a)$$

- **Update** with an observation: receive o, and keep only the states consistent with it.

$$b_o = UPDATE(\hat{b}, o) = \{s : o = PERCEPT(s) \; and \; s \in \hat{} \; b\}$$

- The two steps together:

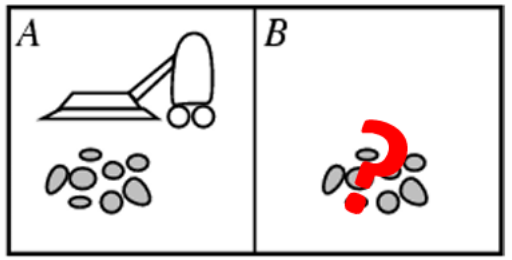
$$b \leftarrow Update(Predict(b, a), o)$$



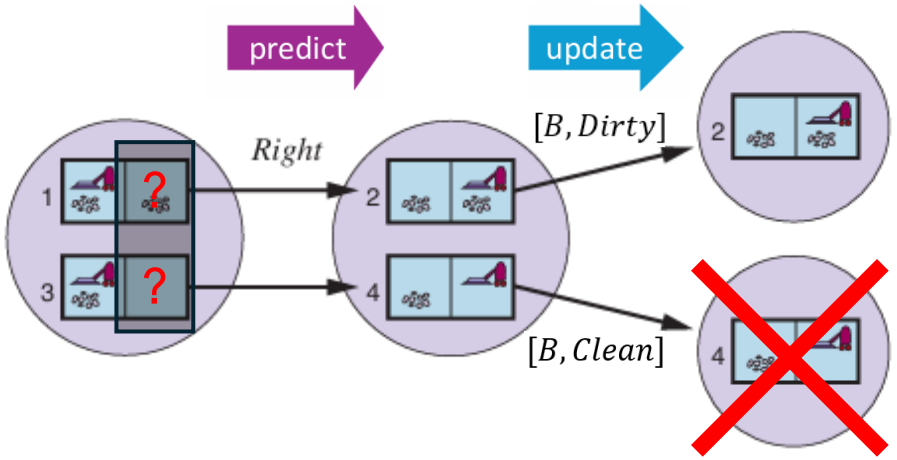
- The agent chooses an action and then receives an observation, so predict always comes before update

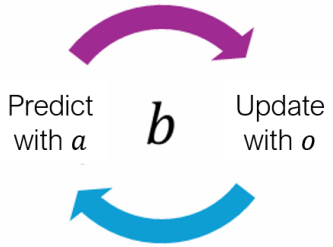
Example: The Local-Sensing Vacuum World

- The agent senses its own square only (Partial Observable): position and whether that square is dirty.



- Predict with $Right$ from $b = \{1, 3\}$, then update with the observation $[B, Dirty]$.
  - $= Update(Predict(\{1, 3\}, Right), [B, Dirty]) = \{2\}$

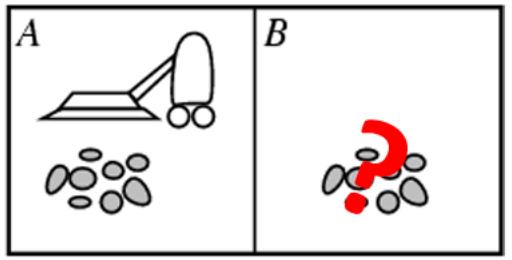


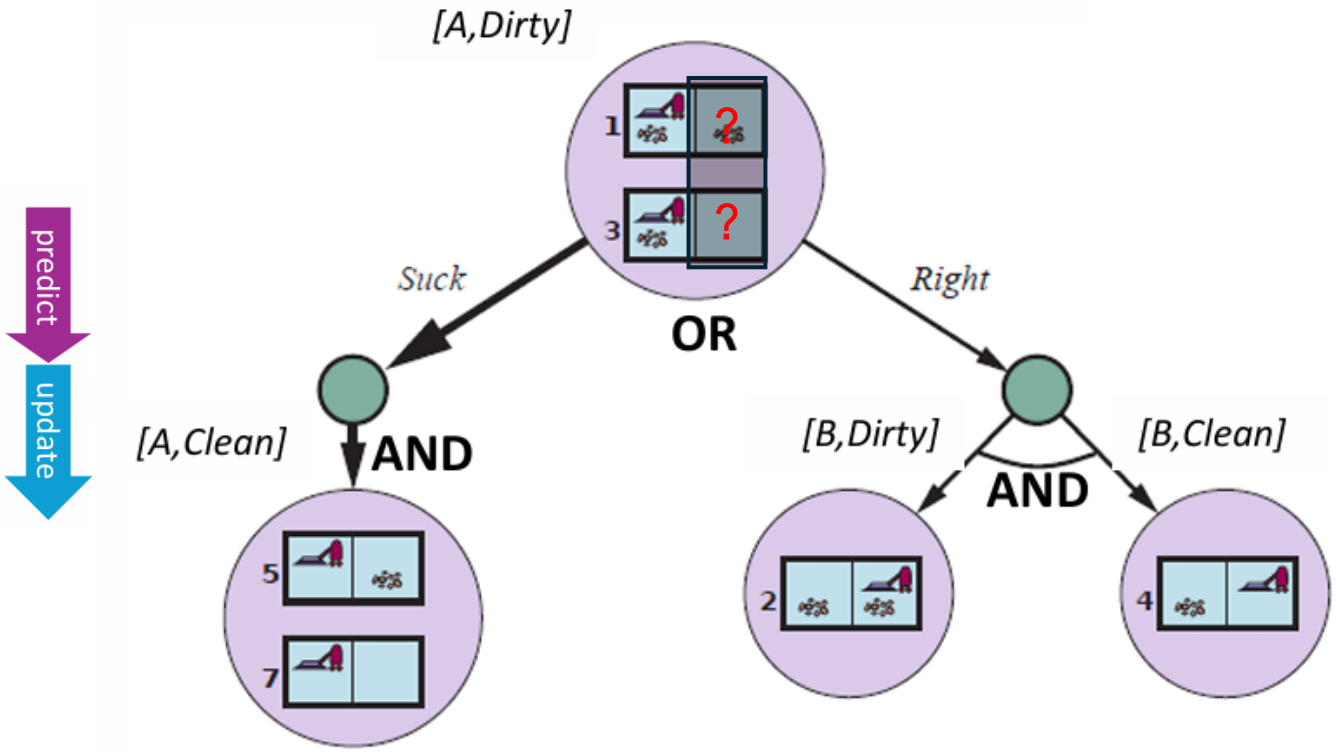


- Two possibilities collapse to one, because only state 2 is consistent with seeing B dirty

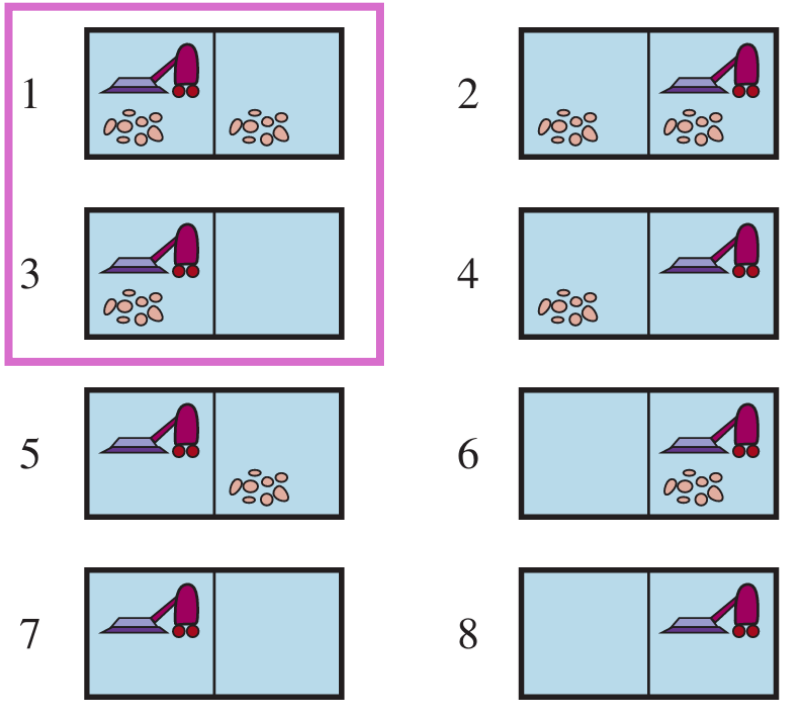
Solving a Partially Observable Problem

- Build ***a conditional plan*** using an AND-OR tree over belief states.
  - The OR node is the agent's choice of action.
  - The AND branches are the observations it might then perceive.





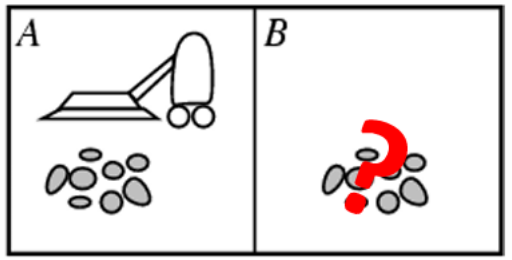
Initial belief state

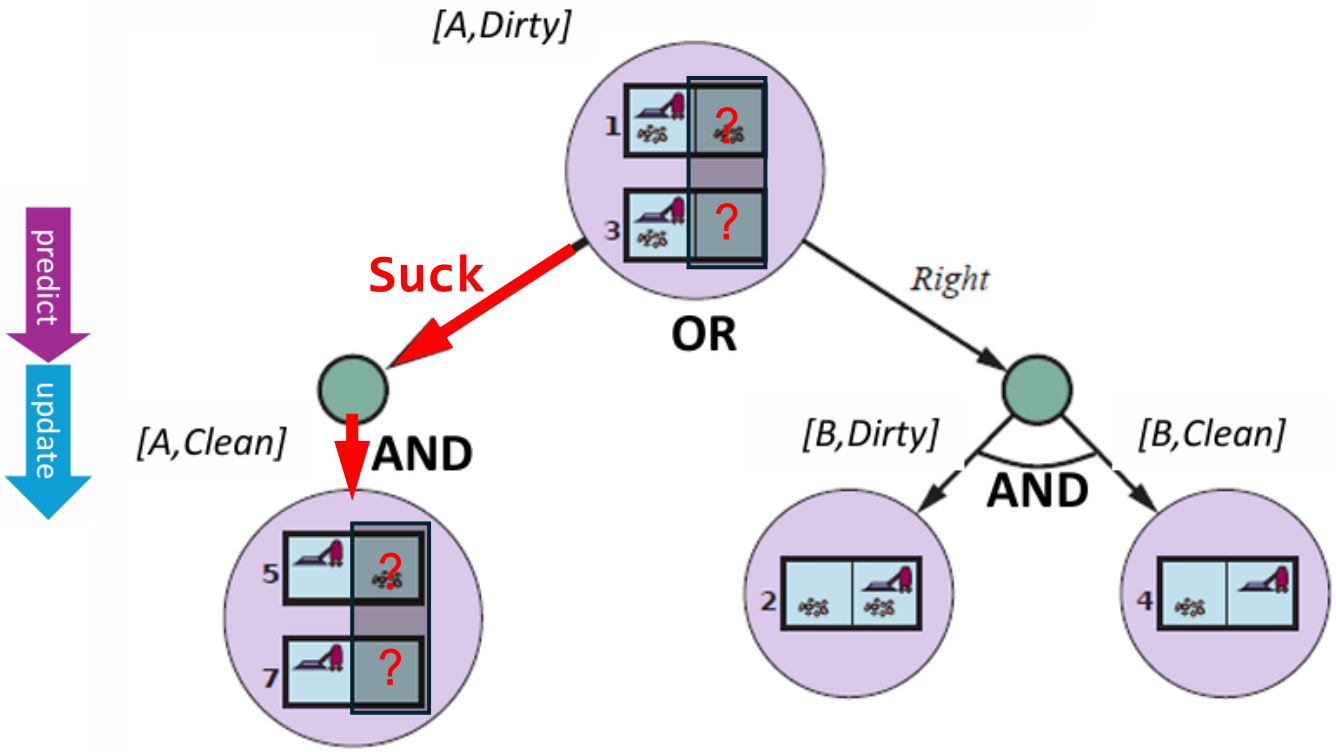


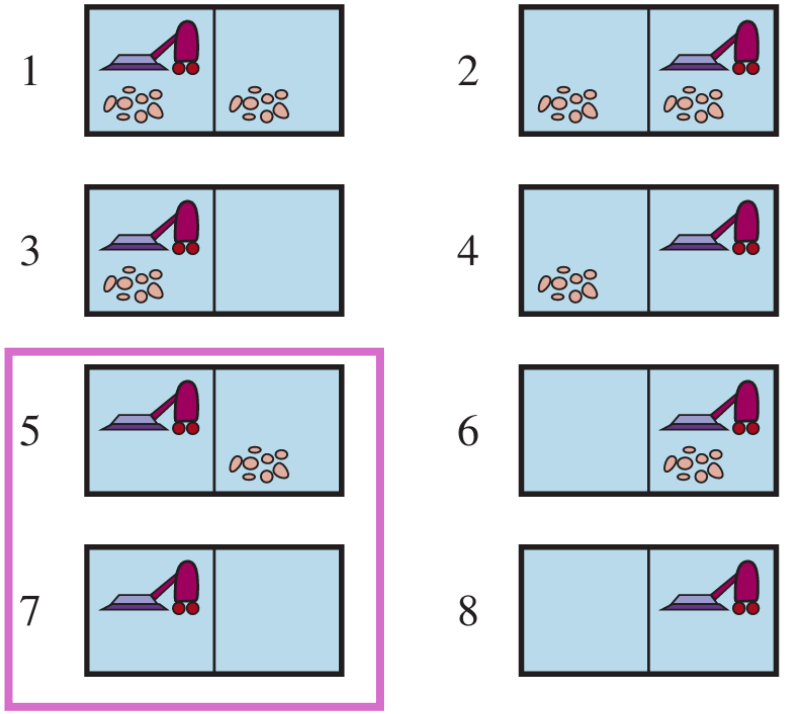
Plan:

Solving a Partially Observable Problem

- Build ***a conditional plan*** using an AND-OR tree over belief states.
  1. Take Suck at the root.







Plan:[Suck, …]

An Agent for Partially Observable Environments

- The problem-solving agent formulates the problem, calls a search algorithm such as AND-OR-SEARCH to get a solution, and executes it

- The solution is **a conditional plan**, not a sequence of actions
  - While acting and receiving percepts, the agent has to keep track of its own belief state:
    $$b \leftarrow Update(Predict(b, a), o)$$
  - That process is called **monitoring**, **filtering** or **state estimation.**

- The difficulty: the agent must update its belief state in real time.
  - In many real applications there is only time to compute an approximate belief state - approximations of this kind are common in control theory and in reinforcement learning

Introduction to Artificial Intelligence

Chapter 4 - Part III: Online Search

Namyup Kim

Soongsil University



Recap: Offline Search

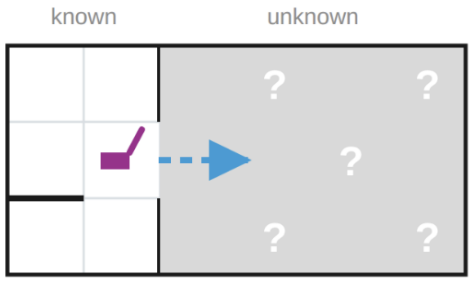
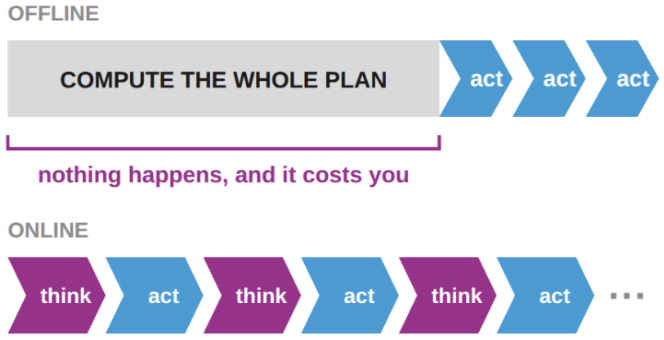
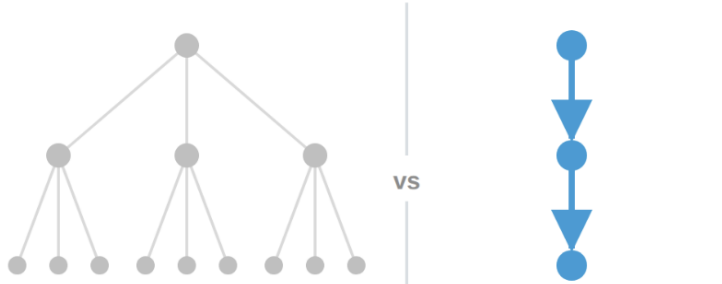
- **Offline search** (also called **planning):** builds a plan using the state space and the transition model, before taking any action.
- The plan may be
  - a sequence of actions,
  - a conditional plan that uses observations to handle uncertainty and incomplete perception.
- The agent predicts the outcome of its actions by searching with a known transition function.
- **The problem**: in an unknown environment, the transition function is not known, so outcomes cannot be predicted and offline search cannot build a plan at all.

Online Search

- Online search does no planning. It searches the real world one action at a time: offline predict and update become act and observe



- When it is useful:

Unknown environments	Real-time problems	Nondeterministic domains
 RESULT(s, a) = ? acting is how you find out	 a little thinking, then move, repeat	 plan for every outcome size grows like n^d handle what happened one path, one step at a time

Implementation: The Online DFS Agent

- Setting: Deterministic and fully observable (the percept is the state), but $result(s, a)$ is unknown.
- Strategy: Try every action, record $result(s, a) \rightarrow s'$, backtrack when all possible actions are searched.

```
agent = OnlineDFSAgent(problem)
s_prime = env.initial_percept()

while True:
    a = agent(s_prime)
    if a == STOP:
        break
    s_prime = env.execute(a)
```

function ONLINE-DFS-AGENT(*problem*, s') **returns** an action
  persistent: s, a, the previous state and action, initially null
    result, a table mapping (s, a) to s', initially empty
    untried, a table mapping s to a list of untried actions
    unbacktracked, a table mapping s to a list of states never backtracked to

  if *problem*.IS-GOAL(s') **then return** *stop*
  if s' is a new state (not in *untried*) **then** *untried*[s'] ← *problem*.ACTIONS(s')
  if s is not null **then**
    result[s, a] ← s'
    add s to the front of *unbacktracked*[s']
  if *untried*[s'] is empty **then**
    if *unbacktracked*[s'] is empty **then return** *stop*
    a ← an action b such that *result*[s', b] = POP(*unbacktracked*[s'])
  else a ← POP(*untried*[s'])
  $s \leftarrow s'$
  return a

Learn transition function

"Untried"=="Frontier"

Stack the previous states.

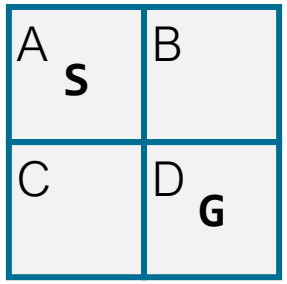
Store (learn) the found transitions

Remember where we came from (to backtrack later).

Action b is Backtrack!

A Example: Exploring a 2x2 World

- The agent must explore an unknown 2x2 space. It starts at the top left and the goal is the bottom right
  - States: A (start), B, C, D (goal)
  - Actions: Up, Down, Left, Right.
  - There are no walls, ***the agent perceives only its current position, and it does not have the map.***

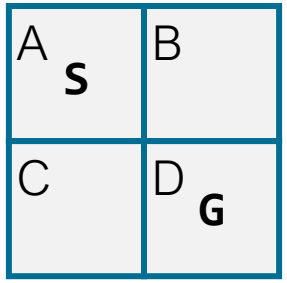


  - Initialization of variables

Variable	Meaning	Initial Value
s, a	Previous state and action	Null
result[s, a]	Table of learned transitions (s, a) → s′	Empty table
untried[s]	Actions not yet tried in state s	None (filled when s is first visited)
unbacktracked[s]	Predecessor states not yet backtracked to from s	None (empty)

The Exploration, Step by Step

- Step 1: the agent is at A
  - untried[A] ← [Right, Down]
  - no previous state, so nothing goes into result
  - A is not the goal; pop Down and move: A → C
- Step 2: the agent is at C
  - C is new, so untried[C] ← [Up, Right]
  - the previous state and action were A and Down, so result[A, Down] ← C
  - unbacktracked[C] ← [A]: the agent can get back to A
  - pop Right and move: C → D
- Step 3: the agent is at D
  - IsGoal(D) is true, so the search stops. The goal is reached

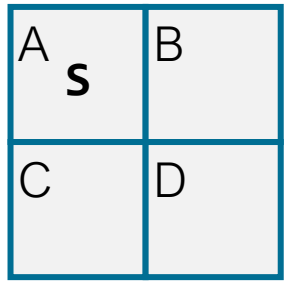


The Exploration Trace

- Total exploration trace summarization

Step	State s'	untried[s']	Action a	result[s, a]	unbacktracked[s']
1	A	[Right, Down]	Down	—	—
2	C	[Up, Right]	Right	(A, Down) = C	[A]
3	D	—	—	(C, Right) = D	[C]

- Example of a backtracking scenario: There is no goal.
  - In state D, do possible actions (= untried[D] become empty).
  - Then, b = pop(unbacktracked[D]) = C.
  - Coming back C, untried[C] has [Up]. Result(C, Up) = A
  - Coming back A, untried[A] has [Right]. Result(A, Right) = B
  - …



- In short, the algorithm repeats the cycle:
  Explore → Dead End → Backtrack → Explore a New Path.

Summary: What Chapter 4 Removed

- Chapter 3 assumed the world is observable, deterministic and known. Chapter 4 removed those assumptions one at a time.
- The path does not matter, only the state: local search: hill climbing, simulated annealing, local beam search, genetic algorithms
- The space is continuous: discretise it, or follow the gradient
- The action outcome is not unique: **belief states** and AND-OR search, with a conditional plan for a solution
- The state is not observable: the same **belief states**, now narrowed by predict and update
- The model itself is unknown: online search, which learns result(s, a) by acting and backtracking